

Equivariant Reassembly of Fractured Objects

A technical description of the data, the network, the losses and the training procedure

October 2026

Abstract

This report describes, from the raw data to the trained model, a two-stage method for reassembling fractured 3D objects. Stage one is a neural network that is *equivariant* to rotations: given the fragments of a broken object, each turned by an unknown random rotation, it predicts the rotation that returns every fragment to its place, together with a rotation-invariant descriptor of every surface point. Stage two uses no learning; it matches descriptors across fragments and solves a small least-squares problem for the translations. Its vector stream is built entirely from vector neurons, and it alternates two kinds of attention layers: a graph-attention layer that runs along the edges of each fragment’s own mesh, and a transformer-style layer that lets fragments exchange information through tokens sampled on their fracture surfaces, under a constraint that keeps each fragment’s output equivariant to its own pose and invariant to the poses of the others. Every quantity is defined mathematically, every design decision is given its reason, and every claim of equivariance is proved and checked numerically. Throughout, the description follows the implementation of the sixth design iteration exactly; where a value was chosen for a particular training run, both the default and the value used are stated.

Contents

1	Overview	4
2	Notation and conventions	5
3	The data	6
3.1	The Breaking Bad dataset	6
3.2	How a break pattern becomes fragment meshes	6
3.3	Items, epochs and sampling	7
3.4	Size of the data	7
4	From a scene to the network’s input	7
4.1	Centring and normalisation	9
4.2	The random rotations and the label	9
4.3	Node features	10
4.4	Edge features	10
4.5	How the perturbed copy is formed	11
4.6	What the network does and does not receive	11
5	Fracture surfaces: identification and use	11
5.1	The dihedral rule	12
5.2	Exact labels from coincidence	13

5.3	How the labels are used	13
5.4	Tokens	13
6	Equivariance	14
6.1	Definition	14
6.2	Why the network should be equivariant	14
6.3	Several fragments	15
6.4	Rules for building equivariant layers	15
7	Vector-neuron building blocks	16
7.1	VN-Linear	16
7.2	VN-LeakyReLU	16
7.3	VN-LayerNorm	17
7.4	The invariant read-out	17
7.5	The scale gate	18
8	The intra layer: vector-neuron graph attention	18
8.1	From GAT to vector-neuron graph attention	18
8.2	The equations	18
8.3	Why each part is there	20
9	The cross layer: vector-neuron transformer attention	21
9.1	The equations	22
9.2	Why it is built this way	22
10	The whole network	23
10.1	From input to output	23
10.2	The rotation head	24
10.3	The rotation convention	25
10.4	The embedding head	25
10.5	What the network outputs	26
10.6	Size	26
11	What a prediction is compared against: the anchor	26
11.1	Why the absolute rotation is the wrong target	26
11.2	The anchor and the alignment	27
11.3	What the anchor target asks of the network	27
11.4	Where the protocol is used	28
12	The losses	28
12.1	Averaging per fragment	28
12.2	Rotation: the geodesic angle	29
12.3	Position	29
12.4	Normals	29
12.5	Face normals	29
12.6	The correspondence loss	30

12.7 The total loss	30
13 Metrics	30
14 Stage two: solving for the translations	31
14.1 Inputs	32
14.2 Matching	32
14.3 Least squares on the fragment graph	32
14.4 Assembly metrics	33
15 Training	33
15.1 Micro-batches and fragment-weighted accumulation	33
15.2 Optimiser	34
15.3 Learning-rate schedule	34
15.4 Epochs, validation and checkpoints	35
15.5 Numerics and robustness	35
15.6 Configuration	36
16 Design decisions and their reasons	37
A The chance level of the geodesic error	37
B Closed forms of the position and normal terms	38
C Why the cross layer pools before building the matrix	38

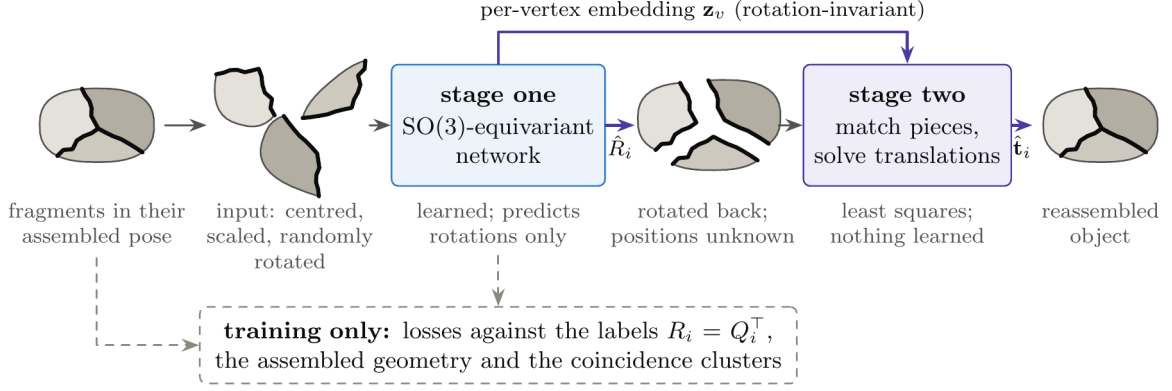


Figure 1: The two stages. The network sees each fragment centred at its own centroid and randomly rotated, and predicts one rotation per fragment ($\hat{R}_i$) and one rotation-invariant embedding per vertex ($\mathbf{z}_v$). Stage two applies the rotations, matches embeddings across fragments, and solves for the translations $\hat{\mathbf{t}}_i$. The labels used by the losses exist only during training.

1 Overview

The task. An object has been broken into F fragments. Each fragment is given as a triangle mesh, and each has been moved by an unknown rigid motion: a rotation and a translation. The goal is to recover, for every fragment, the motion that puts it back where it belongs, so that the fragments reassemble into the original object.

The approach in one paragraph. The problem is split into two stages (Figure 1). *Stage one* is a neural network that predicts only the rotation of every fragment. It is built so that rotating an input fragment rotates its prediction in exactly the corresponding way; this property, *equivariance*, is guaranteed by construction rather than learned from augmented data. Besides one rotation per fragment, the network outputs a 32-dimensional, rotation-*invariant* descriptor (an *embedding*) for every vertex. *Stage two* learns nothing: once every fragment is correctly oriented, placing the fragments is a linear least-squares problem. The descriptors of vertices on the fracture surfaces are matched across fragments, and the translations that make matched points coincide are solved for in closed form.

Why two stages. Rotation is the hard part of the problem, and it is the part where symmetry can be exploited: the correct answer for a fragment changes in a known way when the fragment is rotated, so a network that respects this needs no data to learn it. Translation is different. Once the orientations are known, the translations are determined by the requirement that the two sides of every fracture coincide, which is a quadratic energy with a closed-form minimiser. Learning it would spend data and capacity on something geometry already solves. This is the central bet of the design, and the reason the network never sees, and never predicts, a translation.

The two kinds of layer. The network alternates two layer types.

- The **intra-fragment layer** is a graph-attention layer in the style of GAT [2], built from vector neurons [1]. It passes messages along the edges of each fragment’s own mesh, so information moves only within a fragment. In this report it is called the *intra layer* or the *VN graph-attention layer* (Section 8).
- The **cross-fragment layer** is a transformer-style attention layer [3], also built from vector neurons. It connects *tokens*, a fixed budget of vertices sampled on the fracture surfaces, across different fragments of the same object. It is called the *cross layer* or the *VN transformer layer* (Section 9).

Note the assignment: the graph-attention (GAT-style) layer is the *intra* layer, and the transformer-style layer is the *cross* layer.

How to read this report. Section 2 fixes notation. Sections 3 to 5 follow the data from the files on disk to the tensors the network receives, including how fracture surfaces are found. Sections 6 and 7 give the theory of equivariance and the vector-neuron building blocks. Sections 8 to 10 describe the two layers and the whole network, including how its outputs become a rotation. Sections 11 and 12 define what the rotation is compared against and every loss term. Section 14 solves for translations, Section 15 describes training, and Section 16 collects the design decisions and their reasons. The appendices contain derivations.

2 Notation and conventions

Vectors are bold lower case and are columns: a point is $\mathbf{x} \in \mathbb{R}^3$, and a rotation $Q \in \text{SO}(3)$ acts on it as $\mathbf{x} \mapsto Q\mathbf{x}$. A *vector feature* with C channels is a matrix $H \in \mathbb{R}^{C \times 3}$ whose rows $\mathbf{h}_1^\top, \dots, \mathbf{h}_C^\top$ are 3-vectors; rotating every channel gives $H \mapsto HQ^\top$, because $(Q\mathbf{h}_c)^\top = \mathbf{h}_c^\top Q^\top$. All other symbols are listed in Table 1.

Table 1: Notation.

symbol	meaning
S, F, N, E	scenes in a batch; fragments; vertices; directed mesh edges
$f(v), V_f$	the fragment owning vertex v ; the vertices of fragment f
$\mathbf{x}_v$	position of vertex v in the assembled object (world units)
$\mathbf{c}_f, r_f, d$	centroid and radius of fragment f ; the scene divisor $d = \max_f r_f$
$\mathbf{y}_v = (\mathbf{x}_v - \mathbf{c}_{f(v)})/d$	centred, scale-normalised position, assembled pose
$Q_f, R_f = Q_f^\top$	random rotation applied to fragment f ; its label, the rotation that undoes it
$\tilde{\mathbf{p}}_v = Q_f \mathbf{y}_v, \tilde{\mathbf{n}}_v = Q_f \mathbf{n}_v$	the network’s input: perturbed position and normal ($f = f(v)$)
$A_e \in \mathbb{R}^{3 \times 3}$	edge features: two face normals and a relative position
$s_f = \ln r_f$	log-radius of fragment f , the only scalar input
$H \in \mathbb{R}^{N \times C \times 3}, \mathbf{h}_v \in \mathbb{R}^{C \times 3}$	vector features of all vertices; of vertex v
$C, \mathcal{H}, d_h, d_k$	channels; attention heads ($\mathcal{H} = 4$); per-head size in the intra layer ($d_h = 8$) and the cross layer ($d_k = 16$)
$\mathcal{T}, \mathcal{P}(i)$	tokens; the tokens token i attends to (same scene, other fragments)
$M_f \in \text{SO}(3), \hat{R}_f = M_f^\top$	the network’s frame for fragment f and its predicted rotation
$a, \tilde{R}_f$	the anchor fragment of a scene; a prediction aligned to the anchor
$\mathbf{z}_v \in \mathbb{R}^{32}$	the invariant embedding of vertex v
$\langle \cdot, \cdot \rangle, \langle \cdot, \cdot \rangle_F$	Euclidean and Frobenius inner products
$\iota_k(\cdot)$	the invariant read-out with k learned directions (Section 7)
$\theta(A, B), \angle(\mathbf{u}, \mathbf{w})$	geodesic angle between rotations; angle between vectors

Colour code of the figures. Colour always marks the *type* of a quantity: **blue** (solid arrows) is equivariant and rotates with its fragment, **orange** (dashed arrows) is invariant and does not change under any rotation, **green** is attention or communication between elements, and **violet** is an output of the network.

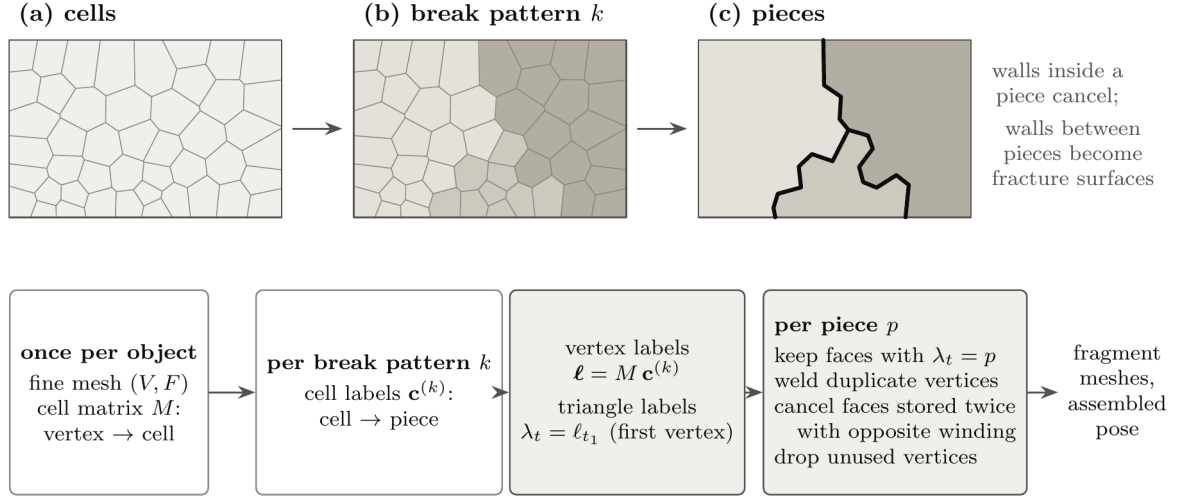


Figure 2: The compressed storage. An object is stored once as a fine mesh that is pre-cut into cells (a); a break pattern assigns every cell to a piece (b); each piece is the union of its cells, the walls between cells of the same piece cancel, and the walls between different pieces remain as fracture surfaces (c). Bottom: the operations that turn one pattern into fragment meshes.

3 The data

3.1 The Breaking Bad dataset

All training and evaluation data come from Breaking Bad [5], a dataset of everyday objects and artefacts fractured by physically based simulation. For every object the dataset provides many *break patterns* (also called fracture modes): different ways the same object breaks, into anywhere from two to about a hundred pieces. The experiments use the *Everyday* subset with the official object split: 407 training objects and 91 validation objects are listed (405 of the training objects were found on disk in the reported run), in 20 categories such as bottles, bowls, mugs, vases and statues. The dataset ships no official test list, so the validation objects are the held-out set. Only the standard break patterns are used (the ones named *fractured*); a second family of patterns stored alongside them is excluded, so that results stay comparable with published numbers.

3.2 How a break pattern becomes fragment meshes

The dataset is stored in a compressed form (Figure 2). For each object there is

- a *fine mesh* with vertices $V \in \mathbb{R}^{n_v \times 3}$ and triangles $F \in \mathbb{N}^{n_f \times 3}$, which is the object pre-cut into n_c small cells; every cell carries its own copy of its boundary triangles;
- a sparse *cell matrix* $M \in \{0, 1\}^{n_v \times n_c}$ that assigns every fine vertex to its cell;
- for every break pattern k , a label vector $\mathbf{c}^{(k)} \in \mathbb{N}^{n_c}$ that assigns every cell to a piece.

A break pattern is turned into fragment meshes in four steps.

1. **Vertex labels.** $\ell = M \mathbf{c}^{(k)}$ gives every fine vertex the piece label of its cell.
2. **Triangle labels.** Each triangle takes the label of its first vertex, $\lambda_t = \ell_{t_1}$; the three vertices of a triangle always belong to the same cell.
3. **Pieces.** Piece p collects the triangles with $\lambda_t = p$.
4. **Clean-up.** Unreferenced vertices are dropped; vertices closer than 10^{-10} are welded into one; then faces that appear more than once are resolved by their winding. Among the faces sharing one vertex set, let c be the number wound one way minus the number wound the other way. If

$c = 0$ the faces cancel; otherwise a single face is kept. Finally, vertices that are no longer used are dropped.

The winding rule is what turns a union of cells into a solid piece. A wall between two cells of the *same* piece is stored twice with opposite orientation and cancels; a wall between two *different* pieces survives once in each piece. Those surviving walls are the fracture surfaces, and because both pieces inherit the same vertices, mating fracture surfaces coincide exactly in the assembled object. Section 5 exploits this.

The result is a list of fragment meshes, all in the *assembled* pose: the coordinate frame in which the object is stored. Patterns that load as a single fragment are skipped (there is nothing to reassemble), as are patterns with non-finite coordinates. Optionally, patterns with more than a given number of pieces can be excluded; with a limit of 20, the benchmark setting used by GARF [6], 26,444 of the 32,396 training patterns and 5,951 of the 7,280 validation patterns remain.

3.3 Items, epochs and sampling

One training *item* is one (object, break pattern) pair. An object has about eighty patterns, and using all of them in every epoch would show the same shape eighty times in a row. Instead, each epoch draws a fixed number of patterns per object (by default 8), with a generator seeded by the object and the epoch, so successive epochs see different patterns of the same object while two runs with the same seed see the same ones.

An epoch is defined as a fixed number of optimiser steps rather than as one pass over the items, because the number of items changes with the patterns drawn per object, the split, the piece-count limit and the number of GPUs, and an epoch is the unit in which validation, checkpoints and the learning-rate schedule are counted. The scenes of an epoch are drawn by concatenating seeded random permutations of the item list, so every item is drawn as evenly as possible.

Category balancing is available but off by default. If it is switched on, item i of object o in category c is drawn with probability proportional to

$$w_i = \begin{cases} m_o^{-\tau} & \text{balance over objects,} \\ (K_c m_o)^{-\tau} & \text{balance over categories,} \end{cases} \quad (1)$$

where m_o is the number of items of object o , K_c the number of objects of category c , and $\tau \in [0, 1]$ interpolates between natural sampling ($\tau = 0$) and full balance ($\tau = 1$).

The *validation set* is fixed for the whole run: its patterns are drawn once, its random rotations are fixed by seeds derived from the scene name, and it is thinned, when requested, to an evenly spaced subset of scenes. A change in the validation numbers therefore reflects a change in the model, never a change in what is measured.

3.4 Size of the data

Table 2 gives an impression of the scale. These figures were measured over the whole dataset (all subsets and all pattern families), not over the Everyday training split alone, so they describe the order of magnitude rather than the exact training distribution.

4 From a scene to the network’s input

A *scene* is one break pattern of one object: F fragment meshes in the assembled pose. This section describes every transformation between that scene and the tensors the network receives.

Table 2: Measured sizes, over all subsets and break patterns of the dataset.

quantity	median	mean	maximum
vertices per fragment	296	1,550	83,039
fragments per break pattern	3	8	99
fracture vertices per fragment (exact labels)	125	164	3,587
share of the surface that is fracture surface: 10.6% of vertices, 6.2% of faces, 7.6% of edges			

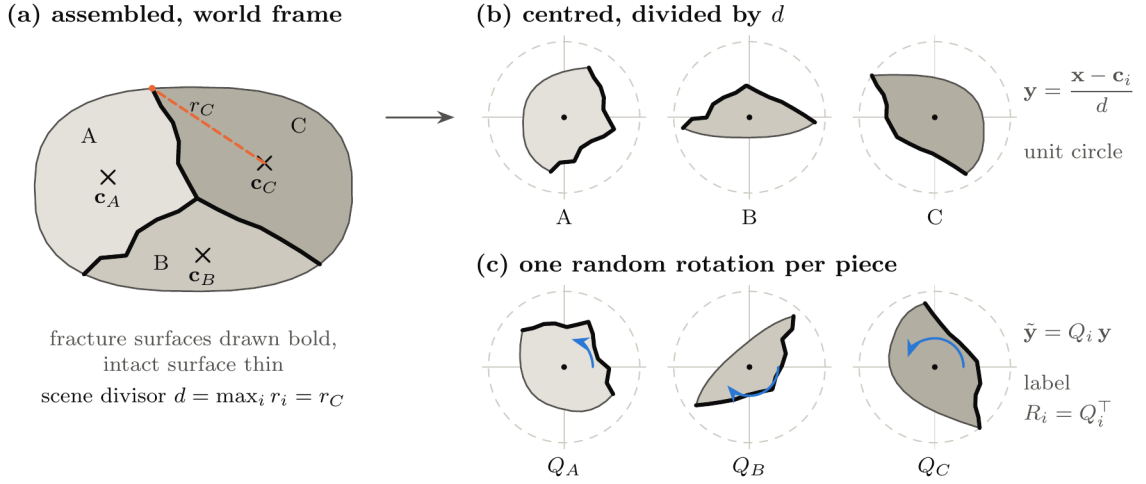


Figure 3: Centring, normalisation and perturbation on a two-dimensional cartoon of three fragments. (a) In the assembled object every fragment has a centroid c_i and a radius r_i ; the largest radius is the scene divisor d . (b) Every fragment is moved to its own centroid and the whole scene is divided by the one number d , so the largest fragment just fits the unit circle and the others keep their relative size. (c) Every fragment is turned by its own random rotation Q_i ; the label is the rotation that undoes it, $R_i = Q_i^\top$.

4.1 Centring and normalisation

For every fragment f the pipeline computes its centroid, its radius and the scene divisor,

$$\mathbf{c}_f = \frac{1}{|V_f|} \sum_{v \in V_f} \mathbf{x}_v, \quad r_f = \max_{v \in V_f} \|\mathbf{x}_v - \mathbf{c}_f\|, \quad d = \max_f r_f, \quad (2)$$

and replaces every vertex by its centred, normalised position

$$\mathbf{y}_v = \frac{\mathbf{x}_v - \mathbf{c}_f(v)}{d} \quad (\text{Figure 3b}). \quad (3)$$

The centroid is the plain mean of the mesh vertices.

Why each fragment is centred on itself. Centring removes the translation completely: whatever offset a fragment had, it is gone after Equation (3). This is deliberate, because stage one predicts rotations only, and translations are recovered in stage two (Section 14), in world units, from the centroids $\mathbf{c}_f$ and the divisor d , which are kept for that purpose. It also makes the perturbation of Section 4.2 a rotation about the fragment’s own centre, so rotating a fragment never moves its centroid.

Why one divisor per scene. Dividing every fragment by its own radius, as some methods do, would make every fragment the same size. Two mating fragments whose sizes differ by a factor of ten would then present fracture surfaces whose scales differ by a factor of ten, although in reality the two surfaces are the same surface. With one divisor per scene, the largest fragment just fits the unit ball, the relative sizes of all fragments are preserved, and mating surfaces keep the same size.

Absolute size as a separate scalar. Normalisation removes the absolute scale of the object, which is informative (a mug is not a vase). It is given back as one scalar per fragment,

$$s_f = \ln r_f \quad (\text{world units}), \quad (4)$$

the only scalar the network receives. The logarithm is taken because fragment sizes are heavy-tailed: fragments range from 4 to 83,039 vertices, and a raw radius would dominate whatever it is combined with. How s_f enters the network without breaking equivariance is described in Section 7.5.

4.2 The random rotations and the label

Every fragment receives its own, independent, uniformly random rotation Q_f . Uniform on $\text{SO}(3)$ means distributed according to the Haar measure, and it is sampled as follows [11]:

1. draw $G \in \mathbb{R}^{3 \times 3}$ with independent standard normal entries and take its QR decomposition $G = QR$;
2. fix the signs, $Q \leftarrow Q \text{diag}(\text{sgn } R_{11}, \text{sgn } R_{22}, \text{sgn } R_{33})$, which makes Q Haar-distributed on the orthogonal group $\text{O}(3)$;
3. if $\det Q = -1$, flip one column, $Q \leftarrow Q \text{diag}(-1, 1, 1)$, which maps the reflections onto the rotations and gives the Haar measure on $\text{SO}(3)$.

The generator is seeded with a stable hash of the scene name, the epoch and the global seed. Consequently a training scene receives new rotations in every epoch, a validation scene keeps the same rotations for the whole run, and every process and every resumed session draws exactly the same numbers.

The network receives the *perturbed* positions and normals

$$\tilde{\mathbf{p}}_v = Q_f \mathbf{y}_v, \quad \tilde{\mathbf{n}}_v = Q_f \mathbf{n}_v \quad (f = f(v)), \quad (5)$$

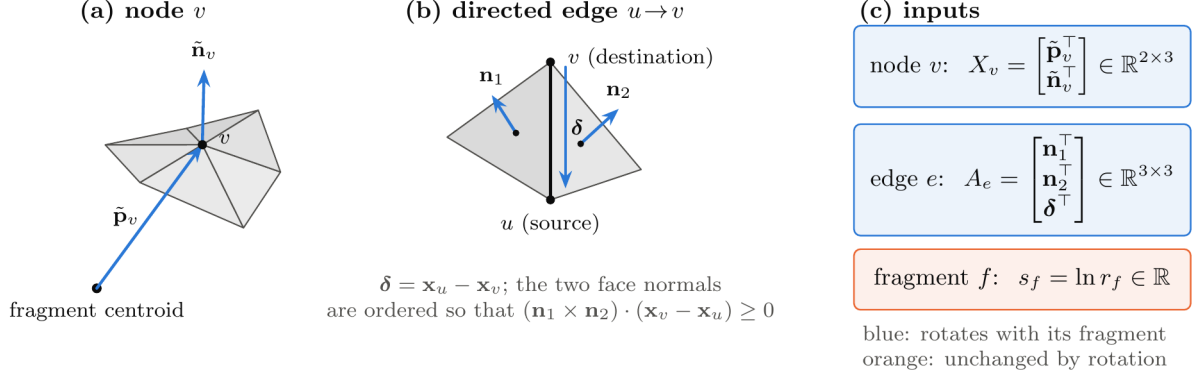


Figure 4: The input features. (a) A vertex carries two vectors: its position relative to the fragment centroid and its normal. (b) A directed edge $u \rightarrow v$ carries three vectors: the normals of its two faces, in an order fixed by the geometry, and the offset $\boldsymbol{\delta}$ from the destination to the source. (c) What the network receives per vertex, per edge and per fragment.

and the label of fragment f is the rotation that undoes its perturbation,

$$R_f = Q_f^\top, \quad \text{so that} \quad R_f \tilde{\mathbf{p}}_v = \mathbf{y}_v. \quad (6)$$

No translation is applied, because centring would remove it again. The divisor d is unchanged by rotations, so normalising and perturbing commute exactly.

4.3 Node features

Every vertex v carries two vectors, stacked as the rows of a 2×3 matrix:

$$X_v = \begin{bmatrix} \tilde{\mathbf{p}}_v^\top \\ \tilde{\mathbf{n}}_v^\top \end{bmatrix} \in \mathbb{R}^{2 \times 3}. \quad (7)$$

The normal is the area-weighted average of the normals of the faces around the vertex,

$$\mathbf{n}_v = \frac{\sum_{t \ni v} (\mathbf{x}_{t_2} - \mathbf{x}_{t_1}) \times (\mathbf{x}_{t_3} - \mathbf{x}_{t_1})}{\left\| \sum_{t \ni v} (\mathbf{x}_{t_2} - \mathbf{x}_{t_1}) \times (\mathbf{x}_{t_3} - \mathbf{x}_{t_1}) \right\|}, \quad (8)$$

since the cross product of two edges of a triangle has twice the triangle’s area as its length. A vertex whose faces are all degenerate receives the zero vector, which every later operation handles without producing invalid numbers.

There are no scalar node features. Every scalar the network uses, such as a distance or an angle, it derives itself, from inner products of vectors, in a way that is invariant by construction (Section 7).

4.4 Edge features

Every fragment’s mesh is its graph: the vertices are the nodes and every mesh edge appears twice, once in each direction. A directed edge $e = (u \rightarrow v)$ has a *source* u and a *destination* v ; messages travel from source to destination. It carries three vectors (Figure 4b):

$$A_e = \begin{bmatrix} \mathbf{n}_1^\top \\ \mathbf{n}_2^\top \\ \boldsymbol{\delta}^\top \end{bmatrix} \in \mathbb{R}^{3 \times 3}, \quad \boldsymbol{\delta} = \mathbf{x}_u - \mathbf{x}_v, \quad (9)$$

with all positions in the normalised coordinates of Equation (3). Here $\mathbf{n}_1$ and $\mathbf{n}_2$ are the unit normals $\mathbf{m}_a, \mathbf{m}_b$ of the two faces that share the edge, in the order fixed by

$$(\mathbf{n}_1 \times \mathbf{n}_2) \cdot (\mathbf{x}_v - \mathbf{x}_u) \geq 0. \quad (10)$$

For the reverse edge $v \rightarrow u$ the two normals swap places and δ changes sign, which satisfies Equation (10) for that direction as well. An edge on the boundary of a fragment has one face, whose normal fills both slots.

Why the order of the two normals is fixed. The two faces of an edge come in no particular order: face numbering has no geometric meaning, and the clean-up of Section 3 reorders faces. A learned linear map treats its input slots differently, so a slot assignment that depended on numbering would feed the network noise. The sign of the triple product in Equation (10) is a geometric property of the edge, and it is unchanged by every rotation, because a rotation preserves cross products and inner products ($\det Q = +1$). The same physical face therefore occupies the same slot in every pose. The sign would flip under a reflection, which is not one of the motions considered.

Why the edge carries the offset δ . In the intra layer (Section 8), the message that a destination receives is a linear function of the source’s features and the edge’s features. Without δ , a message could describe what the neighbour looks like but not in which direction it lies. The raw difference is used rather than a unit direction, because its length, the edge length, is an invariant the network can read through a norm, and normalising would discard it for nothing.

4.5 How the perturbed copy is formed

All geometry is computed once, on the assembled copy: normals, edge features, fracture labels, tokens and coincidence clusters. The perturbed copy is then obtained by rotating it,

$$(\tilde{\mathbf{p}}_v, \tilde{\mathbf{n}}_v, \tilde{A}_e) = (Q_f \mathbf{y}_v, Q_f \mathbf{n}_v, A_e Q_f^\top), \quad (11)$$

rather than by recomputing normals from rotated vertices. The two copies are therefore exact rotations of each other, and cannot differ by round-off in quantities the losses compare directly. In practice only the assembled copy and the labels are transferred to the GPU, and the rotation is applied there.

4.6 What the network does and does not receive

- **Inputs:** $X_v \in \mathbb{R}^{2 \times 3}$ for every vertex, $A_e \in \mathbb{R}^{3 \times 3}$ for every directed mesh edge, and $s_f \in \mathbb{R}$ for every fragment. The first two are equivariant: rotating fragment f by Q turns them into $X_v Q^\top$ and $A_e Q^\top$. The third is invariant.
- **Index structures:** which vertices are tokens and which token pairs may attend to each other (Section 5.4). They decide which vertices the cross layers connect, but carry no numbers.
- **Training labels only:** the rotations R_f , the assembled geometry ($\mathbf{y}_v, \mathbf{n}_v$, assembled edge normals) and the coincidence clusters (Section 5.2).

Scenes in a batch are concatenated; mesh edges never cross fragments and token pairs never cross scenes. In practice one scene is processed per forward pass, because a single scene can hold tens of thousands of vertices.

5 Fracture surfaces: identification and use

The fracture surfaces are where fragments meet, so they are where information should pass between fragments and where stage two looks for correspondences. Two labellings exist: a geometric heuristic that works on one fragment alone, and an exact labelling that needs the assembled object.

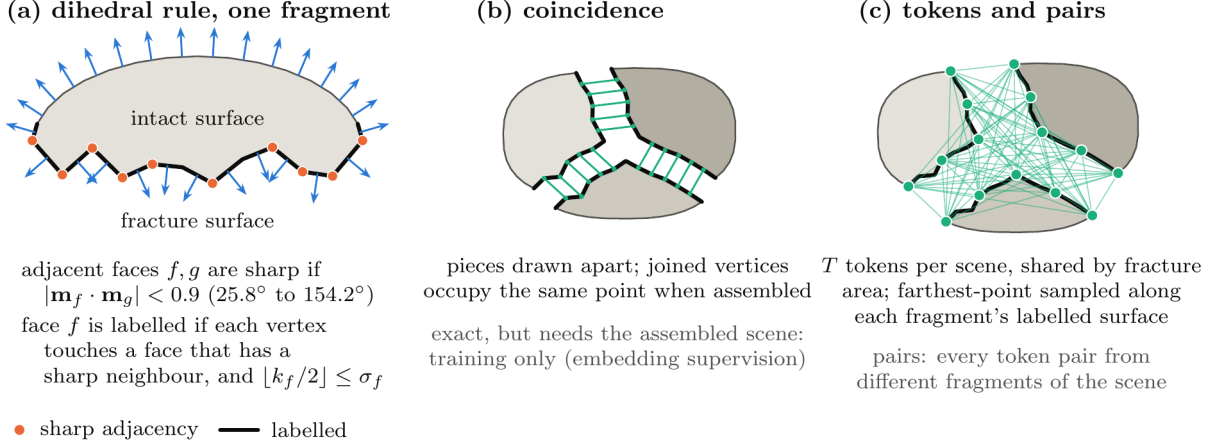


Figure 5: (a) The dihedral rule on a cross-section of one fragment: each segment plays the role of a face and its arrow is the face normal. Adjacent faces whose normals differ by more than 25.8° form a sharp pair (orange dots); faces touching a sharp pair are labelled (bold). The rule finds the jagged fracture surface, plus the two faces at the rims where it meets the intact surface. (b) Exact labels from coincidence: in the assembled object, the two sides of every fracture share their vertices. (c) Tokens: a fixed budget per scene, divided between fragments by fracture area and spread over each fragment's labelled surface by farthest-point sampling; every pair of tokens from different fragments may attend to each other.

5.1 The dihedral rule

Fracture surfaces produced by brittle fracture are rough, while the intact surface of a manufactured object is mostly smooth. The dihedral rule turns this into a test on the angles between neighbouring faces. Two faces f and g are *adjacent* if they share an edge, and with unit face normals $\mathbf{m}_f, \mathbf{m}_g$ the pair is called *sharp* if

$$|\mathbf{m}_f \cdot \mathbf{m}_g| < \tau, \quad \tau = 0.9, \quad (12)$$

that is, if their normals are between $\arccos 0.9 = 25.8^\circ$ and 154.2° apart. Because of the absolute value, nearly opposite normals count as smooth. For face f let k_f be its number of adjacent faces and σ_f the number of sharp pairs it is in. Face f is labelled as fracture surface if

$$(i) \text{ every vertex of } f \text{ lies on some face } g \text{ with } \sigma_g > 0, \quad \text{and} \quad (ii) \lfloor k_f/2 \rfloor \leq \sigma_f. \quad (13)$$

On a closed manifold mesh every face has $k_f = 3$ neighbours, so (ii) asks for at least one sharp neighbour, and (ii) then implies (i). In effect a face is labelled when it touches at least one sharp edge. A face of zero area has a zero normal, so all its pairs count as sharp.

Fallback. A fragment can come out with no labelled face, for instance a very smooth piece of a thin shell. The threshold is then raised step by step for that fragment alone: τ is set just above the q -quantile of the fragment's own values $|\mathbf{m}_f \cdot \mathbf{m}_g|$, for $q = 0.25, 0.5, 0.75, 1$ in turn, until at least one face is labelled. The vertex mask of a fragment is the set of vertices of its labelled faces.

Properties. The rule is computed on each fragment by itself, from absolute inner products of normals, so it is invariant to rotation and needs nothing that would be missing at inference: training and testing see exactly the same labelling. It is also generous. Measured against the exact labels of Section 5.2, it finds 93% of the true fracture faces (recall 0.93) but only 24% of the faces it labels are true fracture faces (precision 0.24), so it over-labels by a factor of about 3.9. The false positives sit where the intact surface itself is sharp: rims, handles, feet and the edges where fracture and intact surfaces meet (Figure 5a shows two such rim faces).

5.2 Exact labels from coincidence

In the assembled object, the two sides of every fracture share vertices (Section 3), so a vertex lies on a fracture surface exactly when another fragment has a vertex at the same position (Figure 5b). Coincidence is found in two passes over the assembled scene: first, coordinates rounded to six decimals are grouped, and every group that spans at least two fragments is marked; second, among the remaining vertices, pairs from different fragments closer than a small tolerance are found with a k-d tree. Joining every coincident pair with a union-find structure groups the vertices into *coincidence clusters*: sets of vertices, from at least two fragments, that occupy one point of the assembled object (where three pieces meet, a cluster has three members).

This labelling is exact, but it needs the assembled object: after the perturbation nothing coincides. It is therefore used only as *supervision* for the embedding (Section 12.6), never as an input.

5.3 How the labels are used

The fracture labels are not fed to the network as a feature. Every vertex remains a node and every mesh edge an edge, whatever its label. The dihedral labels are used in exactly two places: they decide which vertices are eligible to become cross-fragment tokens, and in stage two they decide which vertices are candidates for matching (Section 14).

5.4 Tokens

The cross layers do not connect all vertices of different fragments, which would be quadratic in the number of vertices. They connect a fixed budget of *tokens*: $T = 2048$ vertices per scene, chosen on the labelled fracture surfaces.

Dividing the budget. Let $\mathcal{M}_f$ be the eligible (labelled) vertices of fragment f , $c_f = |\mathcal{M}_f|$ its capacity, and A_f the area of its labelled faces whose three vertices are all eligible. The ideal share is proportional to fracture area,

$$\hat{t}_f = \frac{T A_f}{\sum_g A_g}, \quad t_f = \min\left(c_f, \max(1, \lfloor \hat{t}_f \rfloor)\right) \quad (c_f > 0), \quad (14)$$

so every fragment with an eligible vertex receives at least one token and none receives more than it has. The tokens left over, $T - \sum_f t_f$, are handed to fragments in order of decreasing fractional part $\hat{t}_f - \lfloor \hat{t}_f \rfloor$, each fragment taking as many of them as its capacity allows. If the floors over-spend the budget, the largest allocations are reduced; if the budget is smaller than the number of fragments, the fragments with the largest fracture area receive one token each. The scene receives $\min(T, \sum_f c_f)$ tokens.

Choosing the tokens. Within each fragment, the t_f tokens are chosen by farthest-point sampling with geodesic distances: shortest-path distances along the mesh edges whose two ends are eligible, weighted by edge length. The first token is the eligible vertex farthest from the centroid of the eligible vertices; each subsequent token is the eligible vertex whose distance to the nearest chosen token is largest, ties going to the lowest index. Separate fracture patches are at infinite geodesic distance from each other, so every patch receives a token before any patch receives a second. Geodesic rather than straight-line distance prevents two tokens from counting as far apart merely because the surface folds back on itself.

Why it is done this way. Sampling is done per fragment and from distances alone, so the chosen vertices do not depend on the perturbation; a single farthest-point pass over the whole scene would depend on where the randomly rotated fragments happen to lie. The selection is deterministic, and

it is computed on the assembled copy, but it would choose the same vertices on any rotated copy. Splitting by area gives a fragment with four times the fracture surface four times the tokens.

Pairs. Token i may attend to every token j of the same scene that belongs to a different fragment:

$$\mathcal{P}(i) = \{j \in \mathcal{T} : \text{scene}(j) = \text{scene}(i), f(j) \neq f(i)\}. \quad (15)$$

Tokens of the same fragment are excluded, because the mesh edges already connect them; tokens of other scenes are excluded, because a batch can hold unrelated objects. The number of ordered pairs in a scene is $(\sum_f t_f)^2 - \sum_f t_f^2 < T^2$, about four million at most, whatever the number of fragments: the token budget alone bounds the cost of a cross layer.

6 Equivariance

6.1 Definition

A map f is *equivariant* to a group if transforming the input transforms the output in a corresponding way:

$$f(\rho_{\text{in}}(Q) X) = \rho_{\text{out}}(Q) f(X) \quad \text{for every } Q \in \text{SO}(3), \quad (16)$$

where ρ_{in} and ρ_{out} say how a rotation acts on the input and on the output. When the output does not change at all ($\rho_{\text{out}}(Q)$ is the identity), f is *invariant*. In this work, a rotation acts on a single vector as $\mathbf{x} \mapsto Q\mathbf{x}$, on a stack of vectors $H \in \mathbb{R}^{C \times 3}$ as $H \mapsto HQ^\top$, and on a scalar not at all.

The familiar form $L(Q \cdot \mathbf{x}) = Q \cdot L(\mathbf{x})$ is Equation (16) with the same action on both sides. It holds for the network’s vector features and for its frame, but not for every output: Table 3 lists how each quantity responds when one fragment is rotated by Q , and Figure 6 shows the three cases as commutative diagrams.

Table 3: How every quantity responds when a fragment is rotated by Q .

quantity	symbol	response	kind
input vectors	X_v, A_e	$X_v Q^\top, A_e Q^\top$	equivariant
vector features of every layer	$\mathbf{h}_v \in \mathbb{R}^{C \times 3}$	$\mathbf{h}_v Q^\top$	equivariant
read-outs, attention scores and weights, gates	$\iota_k(\mathbf{h}_v), \lambda, \alpha, \mathbf{g}$	unchanged	invariant
frame	$M_f = [\mathbf{e}_1 \ \mathbf{e}_2 \ \mathbf{e}_3]$	$Q M_f$	equivariant
predicted rotation	$\hat{R}_f = M_f^\top$	$\hat{R}_f Q^\top$	equivariant (from the right)
embedding	$\mathbf{z}_v$	unchanged	invariant
label	$R_f = Q_f^\top$	$R_f Q^\top$	—

6.2 Why the network should be equivariant

The last row of Table 3 is the reason. If the perturbation of a fragment is composed with a further rotation Q , the label becomes $(QQ_f)^\top = R_f Q^\top$: it is multiplied by $Q^\top$ from the right, exactly like an equivariant prediction. The two therefore move together, and the error is independent of the perturbation. Formally, let $\mathbf{x}$ be an assembled fragment and $Q \cdot \mathbf{x}$ its perturbed copy. Then

$$\theta(\hat{R}(Q \cdot \mathbf{x}), R) = \theta(\hat{R}(\mathbf{x}) Q^\top, I Q^\top) = \theta(\hat{R}(\mathbf{x}), I), \quad (17)$$

because the geodesic angle $\theta(A, B)$, defined in Section 12.2, depends only on $A^\top B$, and the product $(AQ^\top)^\top (BQ^\top)$, which equals $Q(A^\top B)Q^\top$, has the same rotation angle as $A^\top B$. The consequences are strong.

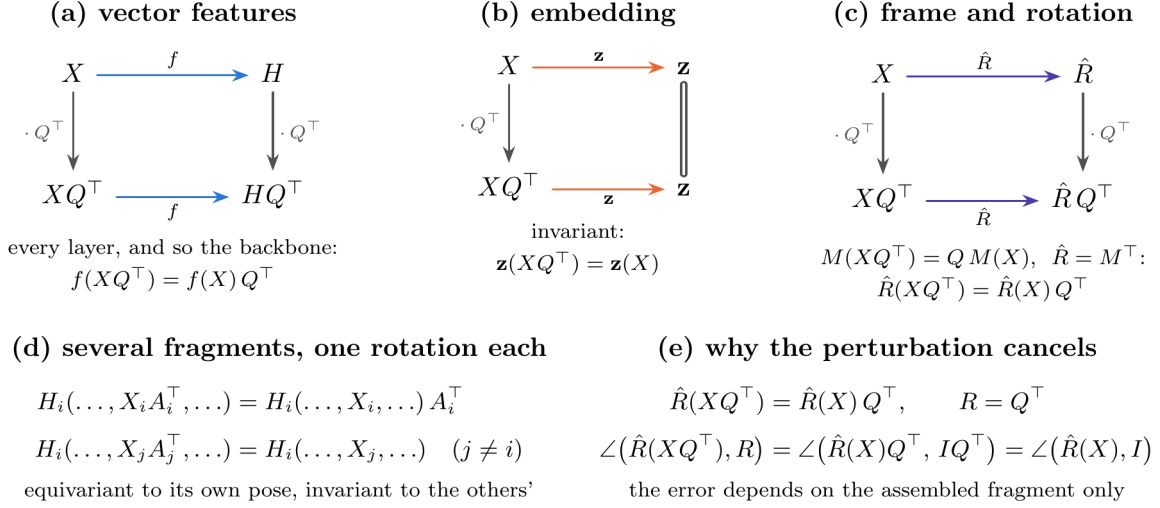


Figure 6: Equivariance in diagrams. (a) Every layer, and therefore the backbone, commutes with rotation. (b) The embedding is unchanged by rotation. (c) The frame rotates and the predicted rotation is multiplied by $Q^\top$ from the right. (d) With several fragments, each fragment’s output follows its own rotation and ignores the others’. (e) Because prediction and label transform the same way, the error does not depend on the random rotation at all.

- The prediction is correct for *every* rotation of a fragment if and only if it is correct for the assembled fragment itself. There is nothing to learn about the perturbation and no need for rotation augmentation.
- The training error of a fragment is a function of its assembled geometry only. The random rotations change the inputs the network sees, but not its loss.
- The network generalises to rotations it has never seen, because there are none it has not, in effect, already seen.

What the network has to learn is a *canonical frame*: a map from an assembled fragment $\mathbf{x}$ to a frame $M(\mathbf{x})$. Sections 10.3 and 11 show exactly which frame the losses ask for.

6.3 Several fragments

A scene has F fragments, each with its own perturbation. Fragment i ’s label, $Q_i^\top$, does not depend on the perturbation Q_j of any other fragment, so a correct output for fragment i cannot depend on Q_j either. The requirement for the vector features H_i of fragment i is therefore

$$\begin{aligned} H_i(\dots, X_i A_i^\top, \dots) &= H_i(\dots, X_i, \dots) A_i^\top && \text{(equivariant to its own pose),} \\ H_i(\dots, X_j A_j^\top, \dots) &= H_i(\dots, X_j, \dots) && \text{(invariant to the pose of every } j \neq i), \end{aligned} \tag{18}$$

which is equation 7 of Wu et al. [4]. A network that could see Q_j would have to learn to ignore it, from data, and Q_j is not weak signal but pure noise injected by the perturbation. Equation (18) makes ignoring it structural. Its practical consequence is a rule for every operation that combines two fragments: only invariant quantities may pass from one fragment to another. An inner product of vector features from two fragments, $\langle A_i \mathbf{a}, A_j \mathbf{b} \rangle$, is not allowed, because it depends on the relative rotation $A_i^\top A_j$. Section 9 shows how the cross layer respects this.

6.4 Rules for building equivariant layers

Every layer of the network is assembled from operations that obey five rules.

1. A composition of equivariant maps is equivariant, so a network of equivariant layers is equivariant.

2. A sum of equivariant features is equivariant.
3. An inner product of two vectors that rotate together is invariant: $\langle Q\mathbf{a}, Q\mathbf{b} \rangle = \langle \mathbf{a}, \mathbf{b} \rangle$. So is a norm.
4. Any function of invariant scalars, including an ordinary neural network with biases and nonlinearities, is invariant.
5. An invariant scalar times an equivariant vector is equivariant: $s(\mathbf{x}) \mathbf{h}(\mathbf{x}) Q^\top$.

Three ordinary habits break equivariance and are therefore absent from the vector stream: a *bias* (a fixed vector does not rotate), a *pointwise nonlinearity* on coordinates (it depends on the axes of the coordinate system), and any *normalisation per coordinate* (dividing the x , y and z components by different numbers is a shear).

Numerical check. The property was verified in double precision on synthetic scenes, by applying an additional random rotation to each fragment and comparing outputs, with the layer schedule of the reported run. The largest deviations were 1.9×10^{-14} for the frame and the rotation, 6.9×10^{-14} for the backbone’s vector features and 1.3×10^{-14} for the embedding. When only one fragment was rotated, the other fragments’ embeddings and frames changed by at most 2.7×10^{-15} and 7.0×10^{-15} , confirming Equation (18). The geodesic error changed by at most 1.8×10^{-14} , confirming Equation (17), and the frames were orthonormal to 2.2×10^{-16} with determinant exactly 1. These are round-off levels: the properties hold exactly, as algebraic identities.

7 Vector-neuron building blocks

Vector neurons [1] replace every scalar feature of an ordinary network with a 3-vector. Other equivariant networks, such as tensor field networks [13] and e3nn [14], carry features of several orders built from spherical harmonics; vector neurons restrict themselves to plain 3-vectors, so that every operation remains a matrix product or an inner product. A layer maps a stack $H \in \mathbb{R}^{C \times 3}$ to a stack $H' \in \mathbb{R}^{C' \times 3}$, acting on the channel axis and leaving the spatial axis to the rotation. Figure 7 shows the five blocks used here.

7.1 VN-Linear

A VN-Linear layer with weight $W \in \mathbb{R}^{C' \times C}$ computes

$$H' = WH, \quad \text{i.e.} \quad \mathbf{h}'_c = \sum_k W_{ck} \mathbf{h}_k. \quad (19)$$

It is equivariant because W acts on the channel axis and Q on the spatial axis: $W(HQ^\top) = (WH)Q^\top$. It has no bias. Weights are initialised uniformly in $\pm 1/\sqrt{C}$, the spatial axis not counting towards the fan-in because it is carried along, not summed over. A wide map applied to stacked inputs can be split exactly into several narrow ones, $W[H_1; H_2; H_3] = W_1 H_1 + W_2 H_2 + W_3 H_3$, because there is no bias; the intra layer uses this.

7.2 VN-LeakyReLU

A pointwise ReLU is not equivariant, because it clips coordinates. The vector-neuron nonlinearity instead learns, for every channel c , a direction $\mathbf{d}_c = (W_d H)_c$ and uses the plane orthogonal to it as the decision boundary:

$$\mathbf{h}'_c = \begin{cases} \mathbf{h}_c & \text{if } \langle \mathbf{h}_c, \mathbf{d}_c \rangle \geq 0, \\ \mathbf{h}_c - (1-a) \frac{\langle \mathbf{h}_c, \mathbf{d}_c \rangle}{\|\mathbf{d}_c\|^2} \mathbf{d}_c & \text{otherwise,} \end{cases} \quad a = 0.2. \quad (20)$$

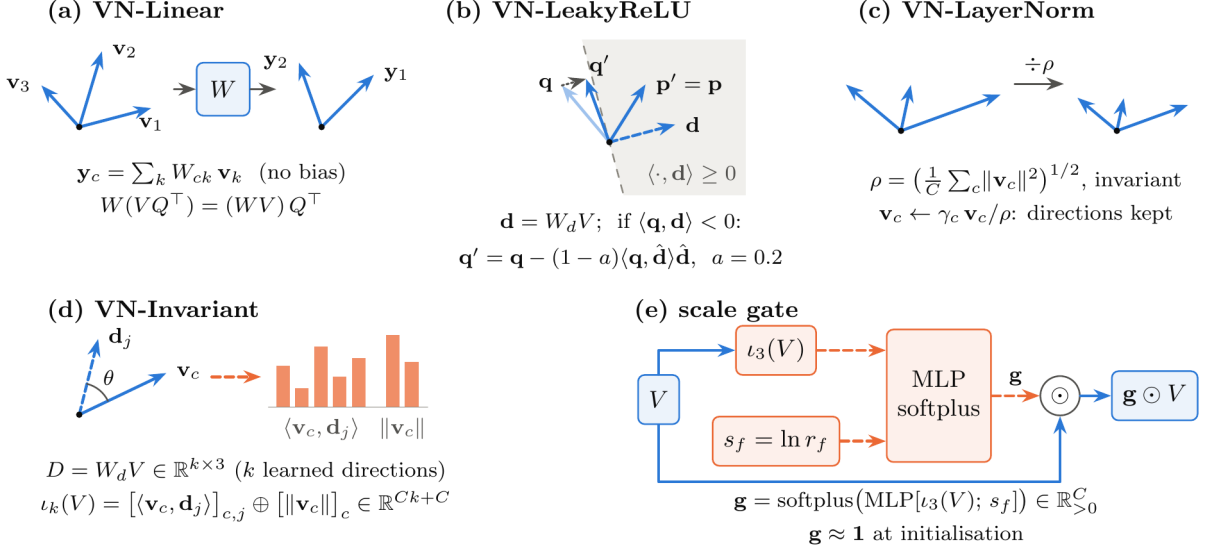


Figure 7: The vector-neuron building blocks. (a) A linear map mixes channels and commutes with rotation. (b) The nonlinearity: a learned direction $\mathbf{d}$ splits space; a vector on its negative side has its component along $\mathbf{d}$ shrunk by $a = 0.2$, one on its positive side passes. (c) Normalisation divides all channels by one invariant number. (d) The invariant read-out: inner products with learned directions, and norms. (e) The scale gate: an invariant, positive gain per channel computed from invariants and the log-radius.

On the negative side the component of $\mathbf{h}_c$ along $\mathbf{d}_c$ is scaled by a ; with $a = 0$ it would be projected onto the plane, with $a = 1$ nothing would happen. The decision is invariant, because $\langle Q\mathbf{h}, Q\mathbf{d} \rangle = \langle \mathbf{h}, \mathbf{d} \rangle$, so the same branch is taken in every pose, and the output is a combination of $\mathbf{h}_c$ and $\mathbf{d}_c$ with invariant coefficients, hence equivariant (rules 3 and 5).

7.3 VN-LayerNorm

Normalisation divides every channel of a vertex by one invariant number, the root mean square of the channel norms, and multiplies by a learned gain per channel:

$$\rho = \left(\frac{1}{C} \sum_{c=1}^C \|\mathbf{h}_c\|^2 + \epsilon \right)^{1/2}, \quad \mathbf{h}'_c = \gamma_c \frac{\mathbf{h}_c}{\rho}. \quad (21)$$

Because ρ is invariant this is an invariant scalar times an equivariant vector. Unlike an ordinary layer normalisation it subtracts no mean and has no bias, and it can never reverse the direction of a channel.

7.4 The invariant read-out

The bridge from vectors to ordinary scalars learns k directions $D = W_d H \in \mathbb{R}^{k \times 3}$ and returns every inner product of a channel with a direction, followed by the channel norms:

$$\iota_k(H) = [\langle \mathbf{h}_c, \mathbf{d}_j \rangle]_{c=1..C, j=1..k} \oplus [\|\mathbf{h}_c\|]_{c=1..C} \in \mathbb{R}^{Ck+C}. \quad (22)$$

Both parts are invariant (rule 3). With $k = 3$ the directions form a learned local frame and the inner products are the coordinates of every channel in that frame; the norms are appended because the inner products recover them only when the directions span space. The network uses $k = 3$ in the scale gate and $k = 4$ in the cross layer and the embedding head.

7.5 The scale gate

The log-radius s_f has no direction, so it cannot be appended to a stack of 3-vectors: there is no legal place to put it. It enters through a gate instead. The invariant description of the vertex and the log-radius of its fragment produce a positive gain per channel:

$$\mathbf{g} = \text{softplus} \left(\text{MLP} \left[\iota_3(\mathbf{h}_v); s_{f(v)} \right] \right) \in \mathbb{R}_{>0}^C, \quad \mathbf{h}'_{v,c} = g_c \mathbf{h}_{v,c}, \quad (23)$$

where the MLP is a linear layer from $4C + 1$ to 64 units, a layer normalisation, a GELU, and a linear layer from 64 to C units. The gain is positive, so the gate can attenuate or amplify a channel but never reflect it. The last layer is initialised with small weights and a bias of 0.5413, for which $\text{softplus}(0.5413) = 1$: an untrained gate is close to the identity. The weights are small rather than zero, because a zero weight would make the gate's output constant and cut off the gradient of everything that feeds it.

Ordinary layers on invariants. Linear layers with biases, layer normalisation, GELU and softmax appear in the network, but only ever on invariant scalars, where rule 4 allows them.

8 The intra layer: vector-neuron graph attention

The intra layer passes information along the edges of each fragment's own mesh (Figure 8). It combines the attention mechanism of graph attention networks with vector neurons: the *score*, which decides how much a vertex listens to each neighbour, is built from invariant inner products; the *message*, which is what it hears, is an equivariant linear map.

8.1 From GAT to vector-neuron graph attention

A graph attention network [2] updates a node with scalar features $\mathbf{h}_v$ as

$$e_{uv} = \text{LeakyReLU} \left(\mathbf{w}^\top [W\mathbf{h}_u \parallel W\mathbf{h}_v] \right), \quad \alpha_{uv} = \text{softmax}_{u \in \mathcal{N}(v)} e_{uv}, \quad \mathbf{h}'_v = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv} W\mathbf{h}_u \right). \quad (24)$$

Three of its parts are not equivariant when the features are vectors: the scalar score $\mathbf{w}^\top[\cdot]$ reads coordinates, the LeakyReLU clips coordinates, and the nonlinearity σ acts on coordinates. The layer used here keeps GAT's structure (a score per edge, a softmax over the incoming edges of a node, a weighted sum of messages) but builds the score from inner products, in the manner of dot-product attention [3], and the update from vector-neuron operations.

8.2 The equations

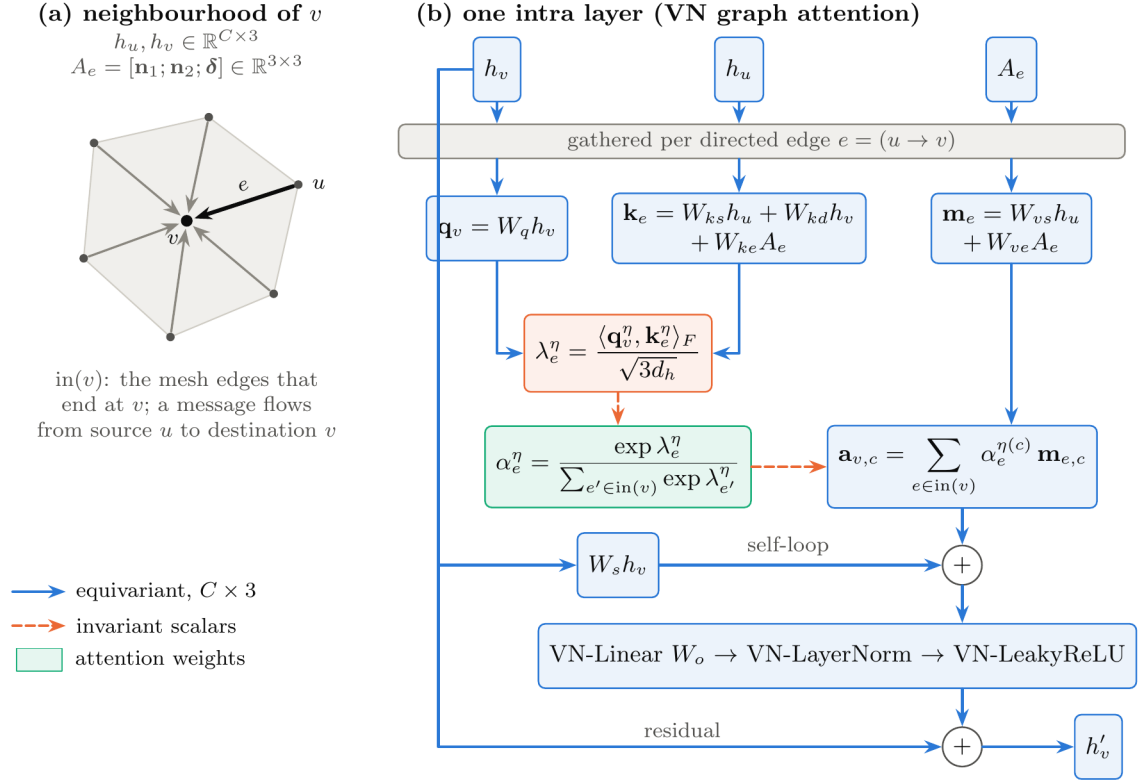
Let $\mathbf{h}_v \in \mathbb{R}^{C \times 3}$ be the features of vertex v and A_e the features of edge $e = (u \rightarrow v)$. With $\mathcal{H} = 4$ heads of $d_h = 8$ vectors each, the layer computes the following.

Query and key. The query is a projection of the destination, and the key mixes the source, the destination and the edge:

$$\mathbf{q}_v = W_q \mathbf{h}_v \in \mathbb{R}^{(\mathcal{H}d_h) \times 3}, \quad (25)$$

$$\mathbf{k}_e = W_{ks} \mathbf{h}_u + W_{kd} \mathbf{h}_v + W_{ke} A_e \in \mathbb{R}^{(\mathcal{H}d_h) \times 3}. \quad (26)$$

All four maps are VN-Linear. The sum equals one VN-Linear map applied to the stacked input $[\mathbf{h}_u; \mathbf{h}_v; A_e]$, because there is no bias; computing it in pieces avoids building that stack for every edge.



$H = 4$ heads: head η scores with its own $d_h = 8$ projected vectors and weights its own block of C/H message channels ($\eta(c)$ owns channel c).

Figure 8: The intra layer. (a) A vertex v listens to the vertices u at the other end of its incoming mesh edges $\text{in}(v)$. (b) The computation for one destination v : equivariant projections (blue), an invariant attention score per edge and head (orange), a softmax over the incoming edges (green), an attention-weighted sum of equivariant messages, a self-loop, a vector-neuron output transform and a residual connection.

Score. Head η owns rows $(\eta - 1)d_h + 1, \dots, \eta d_h$ of the query and key, written $\mathbf{q}_v^\eta, \mathbf{k}_e^\eta \in \mathbb{R}^{d_h \times 3}$. Its score is their Frobenius inner product, scaled by the number of terms:

$$\lambda_e^\eta = \frac{\langle \mathbf{q}_v^\eta, \mathbf{k}_e^\eta \rangle_F}{\sqrt{3d_h}} = \frac{1}{\sqrt{3d_h}} \sum_{r=1}^{d_h} \langle \mathbf{q}_{v,r}^\eta, \mathbf{k}_{e,r}^\eta \rangle. \quad (27)$$

It is invariant: both factors rotate with the fragment, so each inner product does not (rule 3). The scale $1/\sqrt{3d_h}$ keeps the logits of order one, as in dot-product attention, since the inner product sums $3d_h$ products.

Attention weights. For every head, the scores of the edges arriving at v are normalised by a softmax,

$$\alpha_e^\eta = \frac{\exp \lambda_e^\eta}{\sum_{e' \in \text{in}(v)} \exp \lambda_{e'}^\eta} \quad (e \in \text{in}(v)), \quad (28)$$

computed after subtracting the largest score, which leaves the result unchanged and prevents overflow.

Message and aggregation. The message from the source along the edge, and the attention-weighted sum at the destination, are

$$\mathbf{m}_e = W_{vs} \mathbf{h}_u + W_{ve} A_e \quad \in \mathbb{R}^{C \times 3}, \quad (29)$$

$$\mathbf{a}_{v,c} = \sum_{e \in \text{in}(v)} \alpha_e^{\eta(c)} \mathbf{m}_{e,c}, \quad c = 1, \dots, C, \quad (30)$$

where the channels are split into $\mathcal{H}$ contiguous blocks of $C/\mathcal{H}$ and $\eta(c)$ is the head that owns channel c . The heads differ in their scores, and each head's weights scale its own block of message channels; the message map itself is shared, which keeps the parameter count close to that of a plain vector-neuron layer.

Update. The vertex's own features enter through a self-loop outside the softmax; the result passes a VN-Linear map, VN-LayerNorm and VN-LeakyReLU; and the layer is residual:

$$\mathbf{o}_v = \text{VN-LeakyReLU} \left(\text{VN-LN} (W_o(\mathbf{a}_v + W_s \mathbf{h}_v)) \right), \quad \mathbf{h}'_v = \mathbf{h}_v + \mathbf{o}_v. \quad (31)$$

8.3 Why each part is there

- **Invariant scores, equivariant messages.** The two halves of attention have different jobs. The score decides how much to listen and must not change when the fragment turns; the message is what is heard and must turn with the fragment. An invariant weight times an equivariant message is equivariant, and so is the sum over edges.
- **The edge enters both the key and the message.** The key can then score a neighbour by the shape of the surface between the two vertices (the two face normals) and by where the neighbour lies (δ); the message can carry the same information.
- **Self-loop outside the softmax.** A vertex with no incident edges, which occurs in real meshes after clean-up, still produces an output, rather than dividing by an empty sum; and the layer can learn to ignore the neighbourhood entirely.
- **Residual connection and normalisation.** They keep a deep stack of layers trainable and the feature magnitudes controlled, without a bias anywhere in the vector stream.

Proposition 1. *The intra layer is equivariant: replacing every $\mathbf{h}_v$ by $\mathbf{h}_v Q^\top$ and every A_e by $A_e Q^\top$ replaces every $\mathbf{h}'_v$ by $\mathbf{h}'_v Q^\top$.*

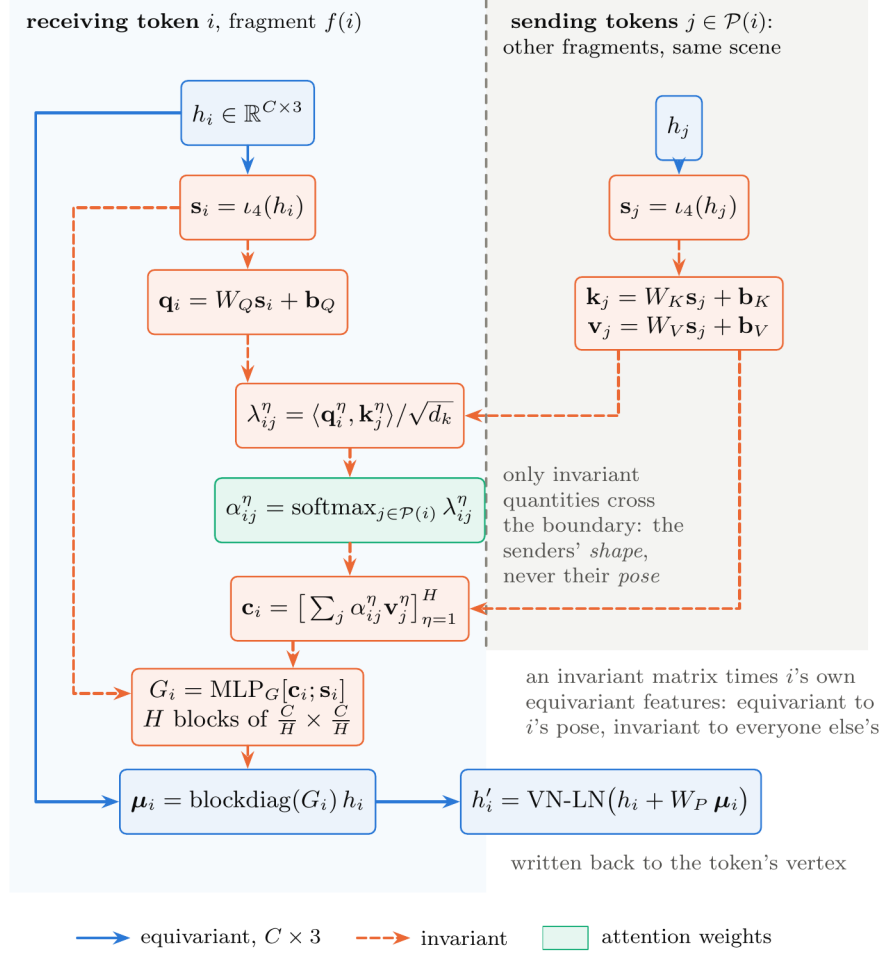


Figure 9: The cross layer for one receiving token i . Every quantity that crosses from the sending fragments (right) to the receiving fragment (left) is invariant (orange); the senders' vectors never cross. The message that token i receives is a matrix built from invariants, applied to i 's own equivariant features.

Proof. Queries, keys and messages are VN-Linear images of rotated inputs, so they are rotated (VN-Linear commutes with Q). The scores are inner products of two rotated stacks, hence unchanged; so are the softmax weights. The aggregate is a sum of rotated messages with unchanged weights, hence rotated, and so is $\mathbf{a}_v + W_s \mathbf{h}_v$. VN-Linear, VN-LayerNorm and VN-LeakyReLU are equivariant (Section 7), and the residual sum of two rotated stacks is rotated. $\square$

Size. The layer has $3CHd_h + 3Hd_h + 4C^2 + 4C$ parameters (query and the two node keys; the edge key; the source message, self-loop, output map and nonlinearity directions; the edge message and the normalisation gains). At $C = 128$ this is 78,432 per layer.

9 The cross layer: vector-neuron transformer attention

The cross layer lets fragments learn about each other. It is a transformer-style attention layer on the tokens of Section 5.4: every token attends to all tokens of the other fragments of its scene (Figure 9). It takes vector features and returns vector features, and it is constrained by Equation (18): a token's output must be equivariant to its own fragment's pose and invariant to every other fragment's pose.

9.1 The equations

Let $\mathbf{h}_i \in \mathbb{R}^{C \times 3}$ be the features of token i (the features of its vertex), with $\mathcal{H} = 4$ heads of size $d_k = 16$.

Invariant descriptions. Each token is first described by invariants, using the read-out of Equation (22) with four directions:

$$\mathbf{s}_i = \iota_4(\mathbf{h}_i) \in \mathbb{R}^{5C}. \quad (32)$$

Scaled dot-product attention on the invariants. Queries, keys and values are ordinary affine maps of the invariant descriptions,

$$\mathbf{q}_i = W_Q \mathbf{s}_i + \mathbf{b}_Q, \quad \mathbf{k}_j = W_K \mathbf{s}_j + \mathbf{b}_K, \quad \mathbf{v}_j = W_V \mathbf{s}_j + \mathbf{b}_V \quad \in \mathbb{R}^{\mathcal{H}d_k}, \quad (33)$$

and every head η attends over the partners $\mathcal{P}(i)$:

$$\lambda_{ij}^\eta = \frac{\langle \mathbf{q}_i^\eta, \mathbf{k}_j^\eta \rangle}{\sqrt{d_k}}, \quad \alpha_{ij}^\eta = \frac{\exp \lambda_{ij}^\eta}{\sum_{j' \in \mathcal{P}(i)} \exp \lambda_{ij'}^\eta}, \quad \mathbf{c}_i = \left[\sum_{j \in \mathcal{P}(i)} \alpha_{ij}^\eta \mathbf{v}_j^\eta \right]_{\eta=1}^{\mathcal{H}} \in \mathbb{R}^{\mathcal{H}d_k}. \quad (34)$$

Up to this point the layer is exactly the attention of a transformer, applied to invariant token descriptions. The biases are allowed because everything here is invariant.

A matrix-valued message. The attended context $\mathbf{c}_i$ and the token's own description $\mathbf{s}_i$ produce one mixing matrix per head,

$$G_i = \text{reshape} \left(\text{MLP}_G [\mathbf{c}_i; \mathbf{s}_i] \right) \in \mathbb{R}^{\mathcal{H} \times \frac{C}{\mathcal{H}} \times \frac{C}{\mathcal{H}}}, \quad (35)$$

where MLP_G is a linear layer from $\mathcal{H}d_k + 5C$ to $2\mathcal{H}d_k$ units, a layer normalisation, a GELU and a linear layer to $\mathcal{H}(C/\mathcal{H})^2$ outputs. Head η 's matrix mixes head η 's block of channels of the token's *own* features:

$$\boldsymbol{\mu}_i = \text{blockdiag} (G_i^1, \dots, G_i^{\mathcal{H}}) \mathbf{h}_i \in \mathbb{R}^{C \times 3}. \quad (36)$$

Update. A VN-Linear projection, a residual connection and VN-LayerNorm complete the layer,

$$\mathbf{h}'_i = \text{VN-LN} (\mathbf{h}_i + W_P \boldsymbol{\mu}_i), \quad (37)$$

and $\mathbf{h}'_i$ is written back to the token's vertex. Vertices that are not tokens are not changed by a cross layer.

9.2 Why it is built this way

- **Scores from invariants.** The natural vector-neuron score, an inner product of vector features of two tokens, would be $\langle A_i \mathbf{a}, A_j \mathbf{b} \rangle = \langle \mathbf{a}, A_i^\top A_j \mathbf{b} \rangle$ after independent rotations A_i, A_j of the two fragments: it depends on their *relative* rotation, which is exactly the noise that Equation (18) excludes. Scores built from the invariant descriptions $\mathbf{s}_i, \mathbf{s}_j$ are independent of both poses.
- **Shape crosses the gap, pose does not.** The sender contributes only invariant quantities: its keys and values. It therefore tells the receiver what the other fragments *look like*, never how they are oriented. Nothing is lost: the relative pose of two randomly rotated fragments is the perturbation, not information about the object.

- **A matrix, not a vector, as the message.** The sender cannot contribute a direction of its own, because a direction would carry its orientation. The most it can contribute is an operator on the receiver’s own vectors. An invariant matrix times the receiver’s equivariant features is equivariant to the receiver’s pose and invariant to everyone else’s. This is the construction of the correlation module of Wu et al. [4], $C_{ij} = G_j F_i$ with G_j an invariant matrix describing the sender and F_i the receiver’s equivariant features. Here the matrix is a full $(C/\mathcal{H}) \times (C/\mathcal{H})$ block per head, so the sender can re-mix the receiver’s channels arbitrarily.
- **Pool first, then build the matrix.** The context is pooled over the partners before the matrix is built, so one set of matrices is computed per receiving token rather than per pair. A scene can have millions of pairs, and a matrix per pair would not fit in memory.
- **Near-identity start.** The last layer of MLP_G is initialised with small weights and zero bias, so an untrained cross layer sends almost no message and passes the token features through, apart from the normalisation, instead of injecting noise into a network that is already doing something. Small rather than zero weights keep the gradient flowing to the attention.

Proposition 2. *The cross layer satisfies Equation (18): rotating token i ’s fragment by A_i multiplies $\mathbf{h}_i'$ by $A_i^\top$ from the right, and rotating any other fragment leaves $\mathbf{h}_i'$ unchanged.*

Proof. The descriptions $\mathbf{s}_j$ of all tokens are invariant to every rotation, so the scores, the weights, the context $\mathbf{c}_i$ and the matrices G_i do not change under any of the rotations. The receiver’s features enter only through $\boldsymbol{\mu}_i = \text{blockdiag}(G_i) \mathbf{h}_i$ and the residual term, both linear in $\mathbf{h}_i$ with invariant coefficients: rotating fragment $f(i)$ multiplies them by $A_i^\top$ from the right, and VN-Linear and VN-LayerNorm preserve this. Rotating another fragment changes neither $\mathbf{h}_i$ nor any invariant, so $\mathbf{h}_i'$ is unchanged. $\square$

Size and cost. The layer has $4C$ parameters for the read-out directions, $3(5C \cdot \mathcal{H}d_k + \mathcal{H}d_k)$ for the queries, keys and values, $(\mathcal{H}d_k + 5C)2\mathcal{H}d_k + 2\mathcal{H}d_k + 4\mathcal{H}d_k + 2\mathcal{H}d_k \cdot \mathcal{H}(C/\mathcal{H})^2 + \mathcal{H}(C/\mathcal{H})^2$ for MLP_G , C^2 for the projection and C for the normalisation. At $C = 128$ this is 758,976 per layer, 70% of it in the last layer of MLP_G . The work is proportional to the number of pairs, at most T^2 per scene (Section 5.4); during training the gathered per-pair quantities are recomputed in the backward pass instead of being stored, which reduces the memory per pair by an order of magnitude at almost no cost in time.

10 The whole network

10.1 From input to output

1. **Lift.** A VN-Linear map lifts the two input vectors of every vertex to C channels, $\mathbf{h}_v = W_{\text{in}} X_v \in \mathbb{R}^{C \times 3}$.
2. **Scale.** The scale gate of Section 7.5 conditions every vertex on the log-radius of its fragment.
3. **Attention layers.** A sequence of intra and cross layers. The default sequence is five intra layers, three cross layers and one more intra layer; the reported run interleaves them as intra, intra, cross, intra, cross, intra, cross, intra.
4. **Heads.** The final features feed the rotation head and the embedding head (Sections 10.2 and 10.4).

Why intra layers must follow cross layers. A cross layer changes only the token vertices; the rotation head averages over *all* vertices of a fragment. Information that arrives at the tokens therefore reaches the rest of the fragment only through intra layers that come after the cross layers, each extending its reach by one ring of mesh neighbours. With the default 2,048 tokens against a

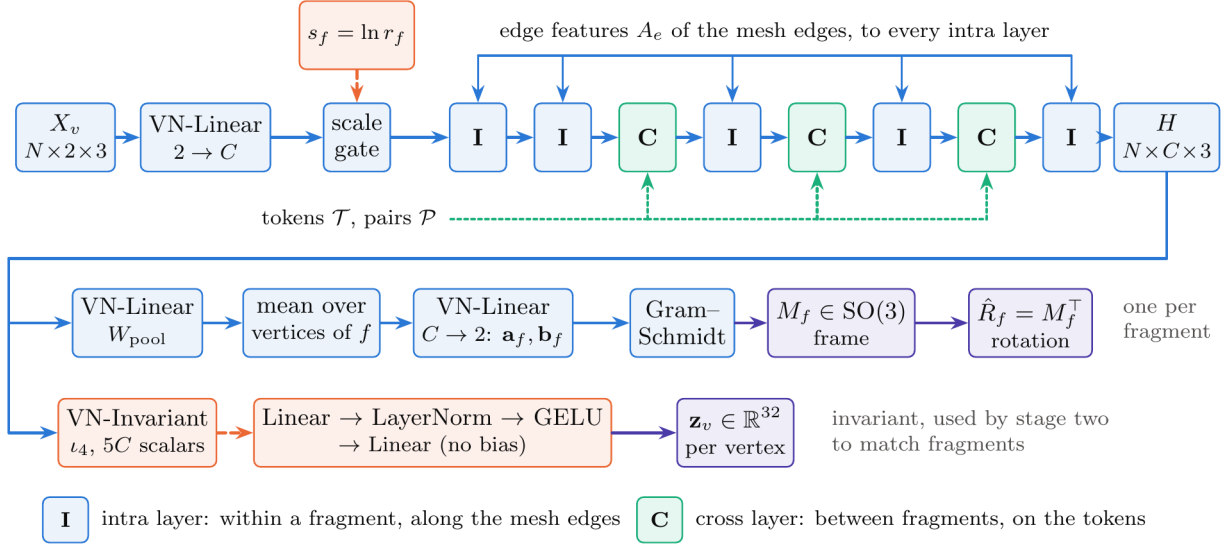


Figure 10: The network as configured for the reported run: an input VN-Linear layer and the scale gate, eight attention layers in the order intra, intra, cross, intra, cross, intra, cross, intra, and two heads. The edge features feed every intra layer and the tokens and pairs feed every cross layer. The rotation head pools each fragment’s vertices into one frame; the embedding head reads out an invariant descriptor for every vertex.

median 9,149 vertices per scene, about 78% of the pooled signal would come from vertices that never heard from another fragment if no intra layer followed the last cross layer. On synthetic scenes the fraction of vertices reached by cross-fragment information was 9%, 51%, 81% and 93% with zero to three intra layers after the last cross layer. Interleaving the two kinds, as in the reported run, also lets every cross layer work with features that already carry the previous cross layer’s information spread over the surface.

10.2 The rotation head

For every fragment f the head computes

$$\mathbf{m}_f = \frac{1}{|V_f|} \sum_{v \in V_f} W_{\text{pool}} \mathbf{h}_v \in \mathbb{R}^{C \times 3} \quad (\text{VN-Linear, then mean over the fragment}), \quad (38)$$

$$\begin{bmatrix} \mathbf{a}_f^\top \\ \mathbf{b}_f^\top \end{bmatrix} = W_{\text{head}} \mathbf{m}_f \in \mathbb{R}^{2 \times 3} \quad (\text{VN-Linear, } C \rightarrow 2), \quad (39)$$

and turns the two vectors into a rotation by Gram–Schmidt orthogonalisation (Figure 11a):

$$\mathbf{e}_1 = \frac{\mathbf{a}_f}{\|\mathbf{a}_f\|}, \quad \mathbf{e}_2 = \frac{\mathbf{b}_f - \langle \mathbf{b}_f, \mathbf{e}_1 \rangle \mathbf{e}_1}{\|\mathbf{b}_f - \langle \mathbf{b}_f, \mathbf{e}_1 \rangle \mathbf{e}_1\|}, \quad \mathbf{e}_3 = \mathbf{e}_1 \times \mathbf{e}_2, \quad M_f = [\mathbf{e}_1 \ \mathbf{e}_2 \ \mathbf{e}_3]. \quad (40)$$

The predicted rotation is the transpose of this frame,

$$\hat{R}_f = M_f^\top. \quad (41)$$

Why this parameterisation. The frame is orthonormal with determinant +1 by construction: the cross product fixes the handedness without a sign test. It is the six-dimensional representation of Zhou et al. [8], which, unlike quaternions or Euler angles, is continuous, and it fits vector features naturally. Rotating the input rotates $\mathbf{a}_f$ and $\mathbf{b}_f$, hence every column of M_f : $M_f \mapsto Q M_f$. A quaternion’s four numbers are coordinates in a fixed basis, and no fixed linear map from vector

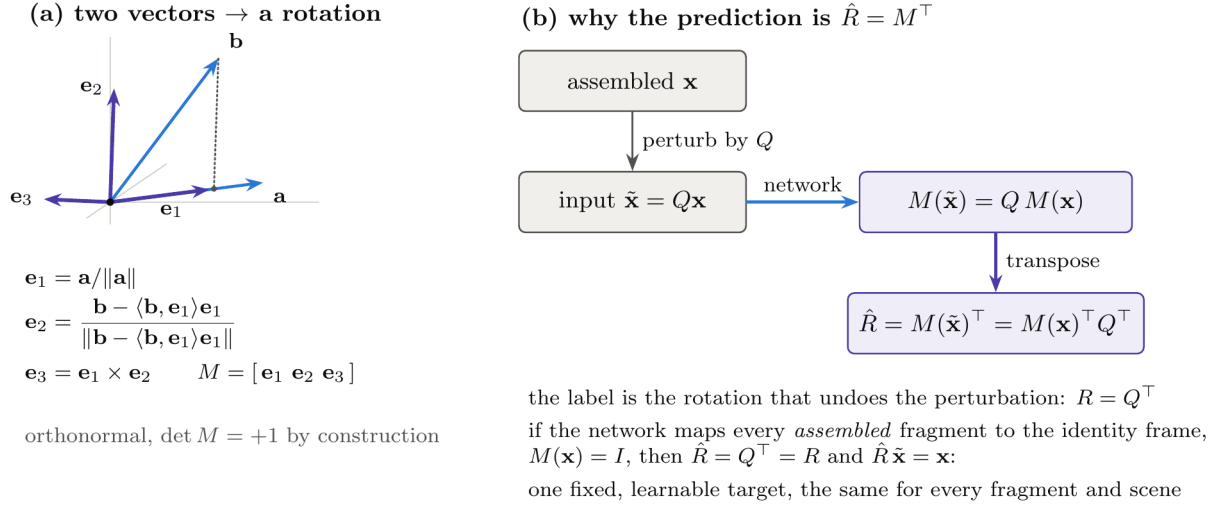


Figure 11: (a) Gram–Schmidt turns two equivariant vectors into a frame: $\mathbf{e}_1$ along $\mathbf{a}$, $\mathbf{e}_2$ the normalised part of $\mathbf{b}$ orthogonal to $\mathbf{e}_1$, $\mathbf{e}_3$ their cross product. (b) The frame of a perturbed fragment is the rotated frame of the assembled fragment, so its transpose undoes the perturbation once the network maps assembled fragments to a common frame.

features can produce them equivariantly. The construction degenerates only when $\mathbf{a}_f$ and $\mathbf{b}_f$ are parallel; the mean of $|\cos \angle(\mathbf{a}_f, \mathbf{b}_f)|$ is logged during training to detect it (0.5 for random directions, near 1 when the frame’s second axis is decided by numerical noise).

10.3 The rotation convention

Why the prediction is $M^\top$ and not M follows from equivariance (Figure 11b). Let $\mathbf{x}$ be an assembled fragment and $Q \cdot \mathbf{x}$ its perturbed copy, and write $M(\mathbf{x})$ for the frame the network assigns to the assembled fragment. Then

$$M(Q \cdot \mathbf{x}) = Q M(\mathbf{x}) \quad \implies \quad \hat{R} = M(Q \cdot \mathbf{x})^\top = M(\mathbf{x})^\top Q^\top. \quad (42)$$

The label is $R = Q^\top$ (Equation (6)). So the prediction is exactly right, for every perturbation, if the network maps assembled fragments to the identity frame, $M(\mathbf{x}) = I$, and then $\hat{R}\tilde{\mathbf{p}}_v = \mathbf{y}_v$ returns every vertex to its assembled position. In general the error is

$$\theta(\hat{R}, R) = \theta(M(\mathbf{x})^\top Q^\top, Q^\top) = \theta(M(\mathbf{x}), I), \quad (43)$$

the angle by which the network’s frame for the assembled fragment misses the identity. This is Equation (17) made concrete: the perturbation has disappeared. Section 11 shows that the loss actually used asks for something weaker and more sensible than $M(\mathbf{x}) = I$.

10.4 The embedding head

For every vertex the head reads out invariants and maps them to a 32-dimensional embedding,

$$\mathbf{z}_v = W_2 \text{GELU} \left(\text{LN} (W_1 \iota_4(\mathbf{h}_v) + \mathbf{b}_1) \right) \in \mathbb{R}^{32}, \quad (44)$$

with $W_1 \in \mathbb{R}^{64 \times 5C}$ and $W_2 \in \mathbb{R}^{32 \times 64}$. It is invariant, because the vertices it will be compared with lie on other fragments, each under its own unknown rotation: equivariant features of two fragments live in unrelated frames and cannot be compared, while invariant ones can. The last layer has no bias, because a constant added to every embedding changes no distance between embeddings and so could never be learned. The embedding is supervised only by the correspondence loss of Section 12.6, and used only by stage two.

10.5 What the network outputs

- a rotation $\hat{R}_f \in \text{SO}(3)$ per fragment, the transpose of its frame M_f , mapping the perturbed fragment towards its assembled orientation; it is used by the losses after alignment to the anchor (Section 11) and by stage two;
- an invariant embedding $\mathbf{z}_v \in \mathbb{R}^{32}$ per vertex, used by the correspondence loss and by stage two;
- the final vector features $\mathbf{h}_v \in \mathbb{R}^{C \times 3}$ and the head’s two vectors $\mathbf{a}_f, \mathbf{b}_f$, kept for diagnostics.

The network never outputs a translation.

Proposition 3. *The whole network is equivariant in the sense of Equation (18): rotating fragment i by A maps $\mathbf{h}_v \mapsto \mathbf{h}_v A^\top$ for $v \in V_i$, $M_i \mapsto A M_i$ and $\hat{R}_i \mapsto \hat{R}_i A^\top$, and leaves every embedding unchanged and every other fragment’s outputs unchanged.*

Proof. The lift is VN-Linear and the scale gate multiplies by an invariant gain. Intra layers connect vertices of one fragment only and are equivariant; cross layers satisfy Equation (18); by induction every layer’s output satisfies it. The pooling is a VN-Linear map followed by a mean over one fragment’s vertices, hence equivariant to that fragment’s pose only; the head is VN-Linear; Gram–Schmidt maps $(A\mathbf{a}, A\mathbf{b})$ to AM because inner products and norms are invariant and $(A\mathbf{u}) \times (A\mathbf{w}) = A(\mathbf{u} \times \mathbf{w})$ for a rotation. The embedding is a function of invariants. $\square$

10.6 Size

Table 4 counts the parameters. Almost all of them sit in the cross layers, in the network that builds the mixing matrices.

Table 4: Parameters, with $\mathcal{H} = 4$, $d_h = 8$, $d_k = 16$ and a 32-dimensional embedding.

component	$C = 128$	count in the reported run
input lift (VN-Linear $2 \rightarrow C$)	256	256
scale gate	41,728	41,728
intra layer, each	78,432	$5 \times 78,432$
cross layer, each	758,976	$3 \times 758,976$
rotation head (pooling map and $C \rightarrow 2$ map)	16,640	16,640
embedding head	43,712	43,712
total		2,771,424
for comparison: default sequence (6 intra, 3 cross) at $C = 128$: 2,849,856; at the default $C = 64$: 928,640		

11 What a prediction is compared against: the anchor

11.1 Why the absolute rotation is the wrong target

The label R_i is the rotation back into the frame in which the object happens to be stored. For an object the network has never seen, that frame cannot be read off the input. Most everyday objects are round, and how far a stored bottle is turned about its own axis is arbitrary: nothing in the fragments reveals it. A network asked for R_i itself can lower that loss on training objects only by remembering how each of them is stored, which does not transfer to new objects. By Equation (43), the absolute target asks for $M(\mathbf{x}_i) = I$: every assembled fragment must be mapped to the dataset’s stored frame.

What the input does determine is how the fragments sit *relative to one another*. The benchmark protocol therefore fixes one fragment at its true pose and scores the others relative to it [6, 7].

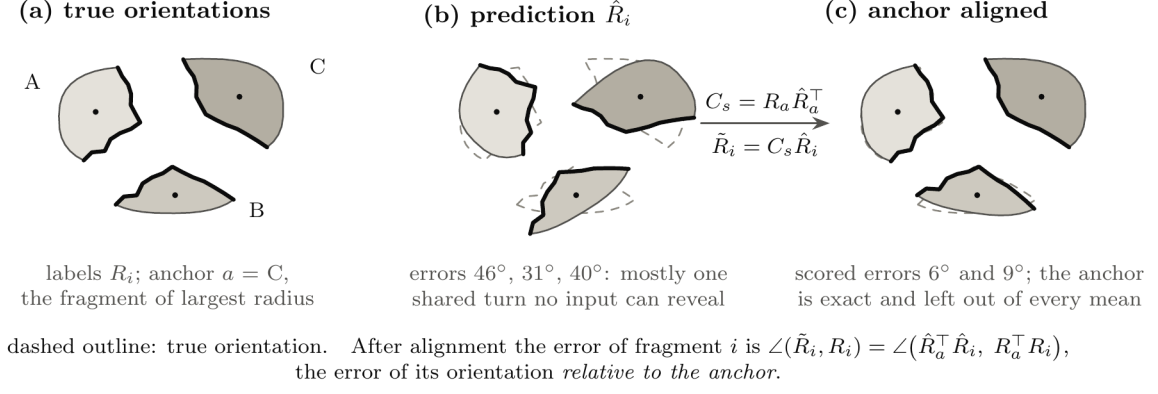


Figure 12: The anchor protocol on the cartoon. (a) True orientations; the anchor is the fragment of largest radius. (b) A prediction that is off mostly by one turn shared by all fragments, which no input can reveal for an unseen, round object. (c) Applying the one rotation C_s that makes the anchor exact removes the shared turn; what remains is each fragment’s error relative to the anchor.

11.2 The anchor and the alignment

In every scene the *anchor* a is the fragment with the largest radius r_f (ties go to the first). It is the same fragment whose radius sets the scene divisor d , its radius is invariant and known at inference, and GARF reports that a random anchor works as well as the largest [6], so the choice is a convention rather than a lever. One rotation per scene is applied on the left of every prediction:

$$C_s = R_a \hat{R}_a^\top, \quad \tilde{R}_i = C_s \hat{R}_i \quad \text{for every fragment } i \text{ of scene } s. \quad (45)$$

Then $\tilde{R}_a = R_a$ exactly, and, because the geodesic angle is invariant under multiplication from either side,

$$\theta(\tilde{R}_i, R_i) = \theta(\hat{R}_a^\top \hat{R}_i, R_a^\top R_i) : \quad (46)$$

the error of fragment i is the error of its orientation *relative to the anchor*. The rotation is applied on the left because, in this convention, a prediction maps the perturbed fragment into the assembled frame, and a global rotation of the assembled result multiplies it on the left. If every prediction in a scene is off by the same global rotation G , so that $\hat{R}_i = G R_i$, then $C_s = G^\top$ and every aligned prediction is exact: the one thing no input determines is removed, and nothing else is. The anchor itself is excluded from every average, since its aligned error is zero by construction; counting it would give every scene a free perfect fragment, a third of the score of a three-fragment scene.

11.3 What the anchor target asks of the network

Combining Equation (45) with the convention Equation (42) gives a remarkably simple picture. Write $M_i = M(\mathbf{x}_i)$ for the frame of the assembled fragment i . Then $\hat{R}_i = M_i^\top Q_i^\top$, so

$$C_s = Q_a^\top (M_a^\top Q_a^\top)^\top = M_a, \quad \tilde{R}_i = M_a M_i^\top Q_i^\top, \quad \theta(\tilde{R}_i, R_i) = \theta(M_a M_i^\top, I) = \theta(M_i, M_a). \quad (47)$$

The anchor-relative error of fragment i is exactly the angle between the frames the network assigns to the assembled fragment i and to the assembled anchor. No perturbation appears. The network is no longer asked to reproduce the dataset’s stored frame, $M_i = I$; it is asked only that *all fragments of one object be mapped to the same frame*, whichever frame that is. Agreeing on a common frame is something the fragments’ shapes, and the information exchanged by the cross layers, can determine for an object never seen before.

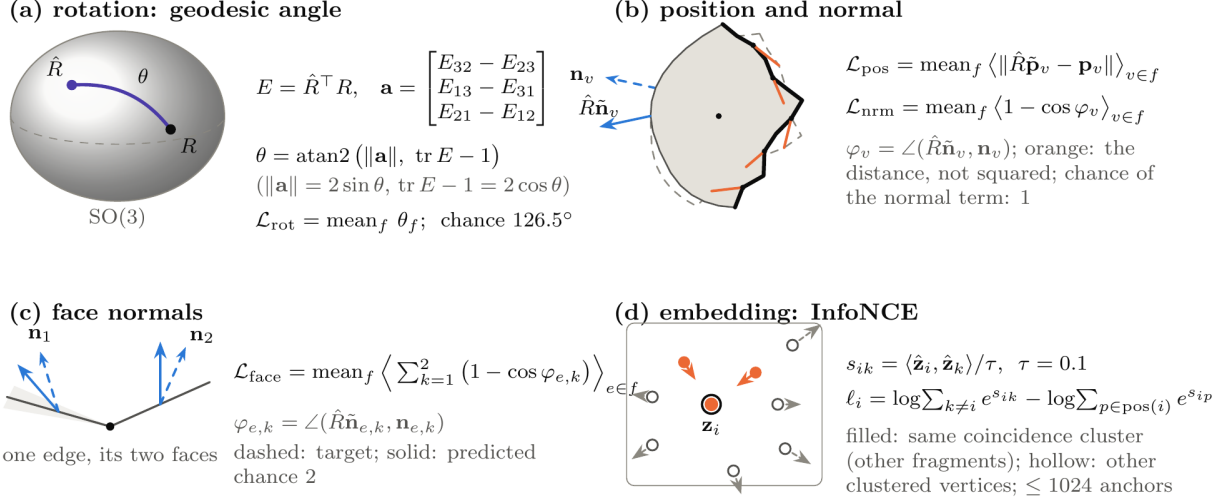


Figure 13: The five loss terms. (a) The geodesic angle between the predicted and the true rotation. (b) The distance between every predicted and true vertex position (orange), and the angle between predicted and true normals. (c) The angles between the predicted and true normals of the two faces of every edge. (d) The correspondence loss pulls together the embeddings of vertices that coincide in the assembled object and pushes apart all others.

11.4 Where the protocol is used

By default the four rotation-dependent losses of Section 12 compare $\tilde{R}_i$, not $\hat{R}_i$, with R_i , and skip the anchors. The alignment is differentiable, including through the anchor’s own prediction, which is therefore trained by every other fragment of its scene. The validation metrics always use the protocol, whatever the training target, with the absolute error reported beside them. The chance level is unchanged, because the relative rotation of two independent uniformly random rotations is itself uniformly random (Appendix A).

12 The losses

12.1 Averaging per fragment

Every rotation-dependent term is first computed per fragment and then averaged over the scored fragments: all fragments under the absolute target, all but the anchors under the anchor target. With $\kappa_f \in \{0, 1\}$ marking the scored fragments,

$$\text{FM}_\kappa(\ell) = \frac{\sum_f \kappa_f \ell_f}{\max(1, \sum_f \kappa_f)}. \quad (48)$$

Terms defined on vertices or edges are first averaged within each fragment, written $\langle \cdot \rangle_{v \in f}$ or $\langle \cdot \rangle_{e \in f}$, so a fragment of 83,039 vertices counts as much as one of 4. In the formulas below, $\tilde{R}_f$ stands for the prediction actually compared: the anchor-aligned one by default, the raw $\hat{R}_f$ under the absolute target.

12.2 Rotation: the geodesic angle

The distance between two rotations is the angle of the rotation that takes one to the other. With $E = \tilde{R}^\top R$,

$$\mathbf{a}(E) = \begin{bmatrix} E_{32} - E_{23} \\ E_{13} - E_{31} \\ E_{21} - E_{12} \end{bmatrix}, \quad \theta(\tilde{R}, R) = \text{atan2}(\|\mathbf{a}(E)\|, \text{tr } E - 1), \quad \mathcal{L}_{\text{rot}} = \text{FM}_\kappa(\theta(\tilde{R}_f, R_f)), \quad (49)$$

which is correct because $\|\mathbf{a}(E)\| = 2 \sin \theta$ and $\text{tr } E - 1 = 2 \cos \theta$ for a rotation by $\theta \in [0, \pi]$. The angle is in radians in the loss and is reported in degrees. The obvious formula, $\arccos((\text{tr } E - 1)/2)$, is not used: its derivative is infinite at $\theta = 0$, exactly where a converging model operates, so its gradient explodes as the fit improves, and clamping its argument to avoid the singularity creates an error floor instead. The two-argument arctangent is smooth at both ends of its range; its derivative at $\theta = 0$ is $1/2$. The norm is computed with a floor of 10^{-20} on its square, which keeps its gradient finite at a perfect prediction without adding a measurable error. For a uniformly random prediction the expected angle is $\pi/2 + 2/\pi$ radians, 126.5° (Appendix A).

12.3 Position

The predicted rotation is applied to the perturbed positions, and the result is compared with the assembled positions:

$$\mathcal{L}_{\text{pos}} = \text{FM}_\kappa\left(\left\langle \|\tilde{R}_f \tilde{\mathbf{p}}_v - \mathbf{y}_v\| \right\rangle_{v \in f}\right). \quad (50)$$

The distance is not squared, and it is measured in the normalised units of Equation (3). Because $\tilde{\mathbf{p}}_v = Q_f \mathbf{y}_v$, the residual is $(\tilde{R}_f Q_f - I) \mathbf{y}_v$, and if the error rotation $\tilde{R}_f Q_f = \tilde{R}_f R_f^\top$ turns by θ_f about the unit axis $\mathbf{u}_f$,

$$\|\tilde{R}_f \tilde{\mathbf{p}}_v - \mathbf{y}_v\| = 2 \sin \frac{\theta_f}{2} \|\mathbf{y}_v - (\mathbf{y}_v \cdot \mathbf{u}_f) \mathbf{u}_f\| \quad (51)$$

(Appendix B). The term is the rotation error weighted by how far the vertices lie from the error axis: a turn about an axis along which the fragment is thin moves little and costs little. It has no universal chance level; for points on the unit sphere and a uniformly random rotation it is $4/3$.

12.4 Normals

$$\mathcal{L}_{\text{norm}} = \text{FM}_\kappa\left(\left\langle 1 - \cos \angle(\tilde{R}_f \tilde{\mathbf{n}}_v, \mathbf{n}_v) \right\rangle_{v \in f}\right), \quad 1 - \cos \angle = 2 \sin^2 \frac{\theta_f}{2} (1 - (\mathbf{n}_v \cdot \mathbf{u}_f)^2). \quad (52)$$

Both vectors are normalised before the cosine is taken. A random prediction scores 1 on average.

12.5 Face normals

Every directed edge carries the normals of its two faces (Section 4.4). Both are rotated by the prediction and compared with the assembled ones; the third edge vector, $\boldsymbol{\delta}$, is not part of this term, because its length is information and normalising it would be wrong:

$$\mathcal{L}_{\text{face}} = \text{FM}_\kappa\left(\left\langle \sum_{k=1}^2 \left(1 - \cos \angle(\tilde{R}_f \tilde{\mathbf{n}}_{e,k}, \mathbf{n}_{e,k})\right) \right\rangle_{e \in f}\right). \quad (53)$$

Both directions of every edge are included; the reverse copy holds the same pair in swapped order, so this equals the mean over undirected edges. A random prediction scores 2.

Why four terms that measure the same rotation. All four depend on the same predicted rotation, so they share their minimiser. They differ in how they weight an error: the geodesic angle counts every error equally, while the position, normal and face terms weight it by the fragment’s geometry, so their gradients emphasise the rotations that visibly move the surface. They also have different profiles near the optimum: the position term behaves like $|\theta|$, the normal and face terms like θ^2 .

12.6 The correspondence loss

The embedding is trained so that vertices which coincide in the assembled object, and only those, have similar embeddings. The positives come from the coincidence clusters of Section 5.2. In every forward pass, a set $\mathcal{A}$ of at most 1,024 clustered vertices is drawn at random as anchors. With unit-normalised embeddings $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|$, a temperature $\tau = 0.1$, and $\text{pos}(i)$ the other members of i ’s cluster in $\mathcal{A}$,

$$s_{ik} = \frac{\langle \hat{\mathbf{z}}_i, \hat{\mathbf{z}}_k \rangle}{\tau}, \quad \mathcal{L}_{\text{emb}} = \frac{1}{|\mathcal{A}'|} \sum_{i \in \mathcal{A}'} \left[\log \sum_{k \in \mathcal{A} \setminus \{i\}} e^{s_{ik}} - \log \sum_{p \in \text{pos}(i)} e^{s_{ip}} \right], \quad (54)$$

where $\mathcal{A}'$ are the anchors with at least one positive in the sample. This is InfoNCE [10] with several positives inside the logarithm. The negatives are all other sampled clustered vertices, which come from the same scene when a forward pass holds one scene: exactly the vertices that stage two will have to tell apart.

Why a contrastive loss. A loss that only pulls coincident vertices together, such as the variance of each cluster about its mean, is minimised by giving every vertex the same embedding. That collapse was observed in practice and is useless for matching, where every distance would then tie. The denominator of InfoNCE pushes non-matching vertices apart and makes collapse expensive. A collapsed embedding scores $\log(|\mathcal{A}| - 1) - \log |\text{pos}(i)|$; an untrained one scores higher still, because the spread of random similarities inflates the denominator, so a falling value early in training is ambiguous. The fraction of anchors whose most similar vertex is a positive, *match@1*, resolves the ambiguity: it is near zero for a collapsed or random embedding and 1 for a perfect one.

12.7 The total loss

$$\mathcal{L} = \mathcal{L}_{\text{rot}} + \mathcal{L}_{\text{pos}} + \mathcal{L}_{\text{norm}} + \mathcal{L}_{\text{face}} + \mathcal{L}_{\text{emb}}. \quad (55)$$

All weights are 1 and untuned: the terms have different natural units (radians, cosines, normalised distances), and choosing weights before seeing how each term moves would be guesswork. Each term is reported separately so that their scales can be measured. No other regulariser is used besides the optimiser’s weight decay.

13 Metrics

- **Geodesic error**, the main number: the mean (and median) of $\theta(\tilde{R}_i, R_i)$ in degrees over all non-anchor validation fragments. The absolute error $\theta(\hat{R}_i, R_i)$ over all fragments is reported beside it.
- **Accuracy at a threshold**: the fraction of scored fragments with an error below 5° , 10° and 30° .
- **Euler-angle error**, for comparison with published tables: the root mean square of the three Euler angles of the residual $\tilde{R}^\top R$ (rotations about the fixed x , y and z axes in turn), averaged over fragments. It is computed on the residual rather than as a difference of Euler angles, which is not a metric on rotations. A 30° error about one axis is 30° geodesic but 17.3° by this measure, so the two must never be confused.

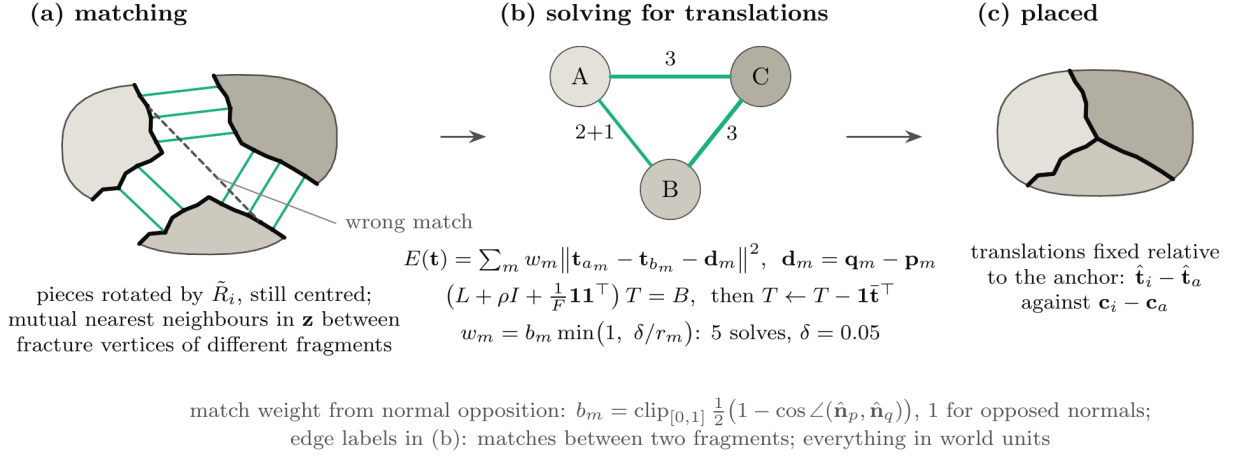


Figure 14: Stage two. (a) With the predicted rotations applied, every fragment still sits at its own centroid. Fracture vertices of different fragments are matched where their embeddings are mutual nearest neighbours; a few matches are wrong. (b) The translations that make matched points coincide solve a least-squares problem on the graph whose nodes are fragments; robust reweighting suppresses wrong matches. (c) The placed fragments.

- **Embedding quality:** match@1 (Section 12.6).
- **Frame health:** the mean $|\cos \angle(\mathbf{a}_f, \mathbf{b}_f)|$ of the head’s two vectors (Section 10.2).
- **Per-category errors:** the geodesic error by object category, on a validation set that is never reweighted.
- **Assembly**, after stage two: the translation error, the Chamfer distance and the part accuracy (Section 14.4).

Table 5: Reference values for a uniformly random prediction, the level every term starts near.

quantity	chance	how it is obtained
geodesic error	126.5° ($\pi/2 + 2/\pi$ rad), s.d. 37.0°	analytic (Appendix A)
geodesic error, anchor protocol	126.5°	relative of two uniform rotations is uniform
axis right, angle about it uniform	90°	analytic; a landmark, not a floor
Euler-angle error	83.2°	Monte Carlo
normal term / face term	1 / 2	$\mathbb{E}[\cos] = 0$ for a uniformly rotated unit vector
position term	$4/3$ for points on the unit sphere	depends on the fragment’s shape
match@1	$\approx \text{pos}(i) / (\mathcal{A} - 1)$	a random nearest neighbour

14 Stage two: solving for the translations

The network predicts orientations only. Once every fragment is correctly oriented, placing it is a well-posed least-squares problem, which this stage solves without learning (Figure 14). It runs at evaluation, not during training, and works in world units, because the benchmark’s thresholds are defined there.

14.1 Inputs

For every vertex v of fragment f , with $\tilde{R}_f$ the anchor-aligned predicted rotation,

$$\mathbf{p}_v = d \tilde{R}_f \tilde{\mathbf{p}}_v, \quad \hat{\mathbf{n}}_v = \tilde{R}_f \tilde{\mathbf{n}}_v, \quad \mathbf{z}_v. \quad (56)$$

If the rotation is exact, $\mathbf{p}_v = \mathbf{x}_v - \mathbf{c}_f$: the fragment is correctly oriented but still centred on its own centroid. Only the translations $\mathbf{t}_f$ are unknown.

14.2 Matching

Candidates. The candidates of a fragment are its dihedral-labelled vertices (Section 5.1), at most 2,048 per fragment, chosen by even striding over the vertex order so that repeated evaluations agree; a fragment with no labelled vertex uses all of its vertices.

Mutual nearest neighbours. For every candidate i , its nearest candidate on any *other* fragment in embedding space is

$$\nu(i) = \arg \min_{j: f(j) \neq f(i)} \|\mathbf{z}_i - \mathbf{z}_j\|, \quad (57)$$

and (i, j) is a match if $\nu(i) = j$ and $\nu(j) = i$. Each point has at most one match; there is no distance threshold. Because the embedding is invariant, the matches do not depend on the predicted rotations at all: the two sides of a fracture can be recognised while the fragments are still apart.

Weights. The two sides of a true interface face each other, so their normals point in opposite directions. Every match $m = (i, j)$ receives the weight

$$b_m = \text{clip}_{[0,1]} \left(\frac{1}{2} (1 - \cos \angle(\hat{\mathbf{n}}_i, \hat{\mathbf{n}}_j)) \right), \quad (58)$$

1 for exactly opposed normals and 0 for aligned ones. This is where the predicted rotations enter the matching.

14.3 Least squares on the fragment graph

With the rotations fixed, the translations that make matched points coincide minimise

$$E(\mathbf{t}) = \sum_m w_m \left\| (\mathbf{p}_{i_m} + \mathbf{t}_{a_m}) - (\mathbf{p}_{j_m} + \mathbf{t}_{b_m}) \right\|^2 = \sum_m w_m \left\| \mathbf{t}_{a_m} - \mathbf{t}_{b_m} - \mathbf{d}_m \right\|^2, \quad \mathbf{d}_m = \mathbf{p}_{j_m} - \mathbf{p}_{i_m}, \quad (59)$$

where a_m and b_m are the fragments of the two matched points. Collecting the translations in $T \in \mathbb{R}^{F \times 3}$ and setting the gradient to zero gives the normal equations

$$LT = B, \quad L = \sum_m w_m (\mathbf{e}_{a_m} - \mathbf{e}_{b_m})(\mathbf{e}_{a_m} - \mathbf{e}_{b_m})^\top, \quad B = \sum_m w_m (\mathbf{e}_{a_m} - \mathbf{e}_{b_m}) \mathbf{d}_m^\top, \quad (60)$$

with $\mathbf{e}_f$ the f -th unit vector. L is the weighted Laplacian of the graph whose nodes are fragments and whose edges are matches. It is singular, because adding one constant vector to every translation changes nothing; this *gauge* freedom is removed by solving

$$\left(L + \rho I + \frac{1}{F} \mathbf{1} \mathbf{1}^\top \right) T = B, \quad \rho = 10^{-6}, \quad (61)$$

and subtracting the mean translation. Since $\mathbf{1}^\top L = 0$ and $\mathbf{1}^\top B = 0$, the solution has zero mean and is the least-squares solution of minimal norm, up to the tiny ridge ρ , which also covers fragments that no match reaches. The system is solved in double precision, because the normal equations of a sparsely connected assembly are ill-conditioned and single precision loses the small fragments first.

Robustness. Some matches are wrong, and a single wrong match can drag a fragment far away. The solve is therefore repeated with weights recomputed from the residuals, a Huber-type iteratively reweighted least-squares scheme [12]:

$$w_m^{(0)} = b_m, \quad r_m = \|\mathbf{t}_{a_m} - \mathbf{t}_{b_m} - \mathbf{d}_m\|, \quad w_m \leftarrow b_m \min(1, \delta/r_m), \quad \delta = 0.05, \quad (62)$$

for five solves in total. Matches with residuals below δ keep their weight; matches far off are down-weighted in proportion to their residual, which approximately minimises the Huber loss of the residuals instead of their squares.

Why the normals are not part of the energy. Normals do not change under translation, so any energy term in the normals would have zero gradient with respect to $\mathbf{t}$. What they do measure is whether a match is plausible, which is how they are used.

Gauge and final pose. The translations are reported relative to the anchor, $\hat{\mathbf{t}}_f - \hat{\mathbf{t}}_a$, and compared with the true offsets $\mathbf{c}_f - \mathbf{c}_a$. The predicted world position of vertex v is $\mathbf{p}_v + \hat{\mathbf{t}}_{f(v)} - \hat{\mathbf{t}}_a$, against the true $\mathbf{x}_v - \mathbf{c}_a$.

Special cases. With no match, every translation is zero. If the match graph falls into several connected pieces, each piece is solved with zero mean on its own and the pieces end up superimposed; the number of matches is reported with every assembly so that a weakly supported result can be recognised. An optional step that pushes apart fragments whose bounding spheres overlap exists but is off, because mating fragments' bounding spheres overlap even in a correct assembly, so it can only trade interface alignment for visual plausibility.

14.4 Assembly metrics

Per scene, over the non-anchor fragments:

- the translation error $\text{RMSE}(T) = \text{mean}_f \sqrt{\frac{1}{3} \|\hat{\mathbf{t}}_f - \mathbf{t}_f\|^2}$, in world units;
- the symmetric Chamfer distance, with squared distances,

$$\text{CD}(\mathcal{X}, \mathcal{Y}) = \frac{1}{|\mathcal{X}|} \sum_{\mathbf{x} \in \mathcal{X}} \min_{\mathbf{y} \in \mathcal{Y}} \|\mathbf{x} - \mathbf{y}\|^2 + \frac{1}{|\mathcal{Y}|} \sum_{\mathbf{y} \in \mathcal{Y}} \min_{\mathbf{x} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\|^2, \quad (63)$$

between predicted and true point sets of at most 2,048 evenly spaced vertices per fragment, for the whole object (anchor included) and for every part;

- the part accuracy: the fraction of parts whose own Chamfer distance is below 0.01.

Every metric is computed per scene and then averaged over scenes.

15 Training

15.1 Micro-batches and fragment-weighted accumulation

One forward pass processes one scene, a *micro-batch*; a scene is a whole set of graphs, often tens of thousands of vertices, and memory is the binding constraint. One optimiser step accumulates K micro-batches. Every loss term is a mean over the n_b scored fragments of micro-batch b , so the backward pass is run on $n_b \mathcal{L}_b$, which turns the mean back into a sum, and the accumulated gradient is divided by the total number of fragments:

$$\mathbf{g} = \frac{\sum_b n_b \nabla \mathcal{L}_b}{\sum_b n_b}. \quad (64)$$

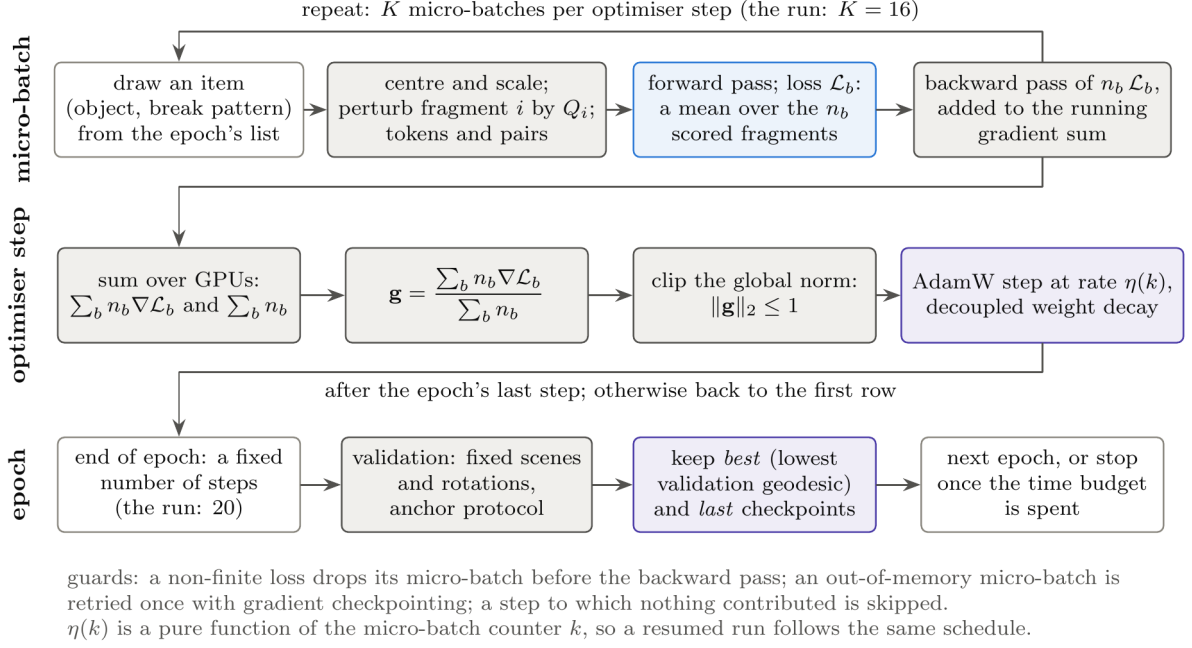


Figure 15: The training procedure: micro-batches of one scene, accumulated into one optimiser step with every fragment weighted equally, and a fixed number of steps per epoch followed by validation and checkpointing.

For the rotation-dependent terms this is exactly the gradient of the mean over every scored fragment in the step, the same as if all K scenes had been processed at once. Dividing by K instead would weight every *scene* equally, although scenes hold anywhere from two to dozens of fragments; on one measured pair of scenes with two and eight fragments, the scene-weighted gradient had a cosine similarity of only 0.80 with the correct one.

Several GPUs. Each GPU processes its own micro-batches. At every step boundary the GPUs sum their fragment counts and their gradients, and every GPU divides the same sum by the same count, so all replicas apply the identical update and never drift apart. Because the step boundary depends only on the position of a micro-batch, every GPU reaches every boundary, whatever micro-batches it had to skip.

15.2 Optimiser

The gradient is clipped to a global Euclidean norm of 1, $\mathbf{g} \leftarrow \mathbf{g} \min(1, 1/\|\mathbf{g}\|)$, and applied with AdamW [9]:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \mathbf{g}, \quad \mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) \mathbf{g}^2, \quad \boldsymbol{\vartheta} \leftarrow \boldsymbol{\vartheta} - \eta(k) \left(\lambda \boldsymbol{\vartheta} + \frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{v}}} + \epsilon} \right), \quad (65)$$

with bias-corrected moments $\hat{\mathbf{m}}, \hat{\mathbf{v}}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, and the weight decay λ applied to every parameter. The decay is decoupled from the gradient: it shrinks every weight by the factor $1 - \eta\lambda$ per step, independently of the adaptive scaling. A step to which no micro-batch contributed is skipped altogether, because AdamW would otherwise still move the weights through momentum and decay.

15.3 Learning-rate schedule

The learning rate is a pure function of the micro-batch counter k , which advances at every micro-batch, including skipped ones, so it is the same on every GPU and after every resume. With

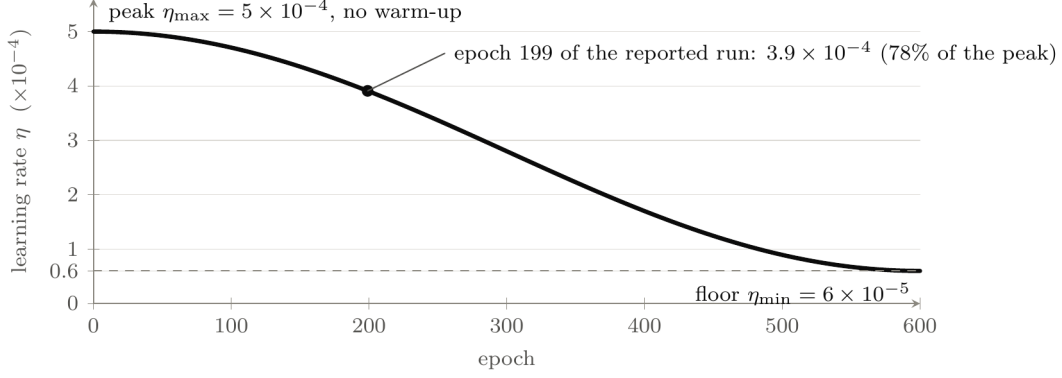


Figure 16: The learning-rate schedule of the reported run: 5×10^{-4} to 6×10^{-5} over 600 epochs of 320 micro-batches, without warm-up.

T_{tot} micro-batches in the whole run, a warm-up of $W = \max(\lfloor wT_{\text{tot}} \rfloor, 1)$ micro-batches, and $p_k = \min((k - W)/(T_{\text{tot}} - W), 1)$,

$$\eta(k) = \begin{cases} \eta_{\max} \frac{k+1}{W}, & k < W, \\ \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos \pi p_k), & k \geq W : \end{cases} \quad (66)$$

a linear warm-up followed by one half-cosine from $\eta_{\max}$ to $\eta_{\min}$, after which the rate stays at the floor. A periodic scheduler is avoided because past its period the rate climbs back up, and because its period is stored in its own state, so changing the number of epochs when resuming might not change the schedule. A step uses the rate of its last micro-batch. By default $\eta_{\max} = 10^{-3}$, the warm-up is 3% of training and $\eta_{\min} = 0.02 \eta_{\max}$. Figure 16 shows the schedule of the reported run.

15.4 Epochs, validation and checkpoints

An epoch is a fixed number of optimiser steps. After every epoch the model is evaluated, without gradients, on the fixed validation set of Section 3, under the anchor protocol. Two checkpoints are kept: the *last* one, written every epoch and also every 30 minutes in the middle of an epoch, and the *best* one, written whenever the validation geodesic error improves. A checkpoint stores the weights, the optimiser state, the step counter, the random-number states, the history and the configuration, so a resumed run continues exactly where it stopped; an interrupted epoch is restarted with the step counter rewound to its beginning. A time budget is checked between epochs: once it is spent, the current epoch is finished, validated and saved, and the run stops.

15.5 Numerics and robustness

- **Single precision.** Training runs in 32-bit floating point. In half precision the attention products of unnormalised vector features overflow, and the softmax then produces invalid numbers; an earlier design of this project also trained measurably worse with mixed precision.
- **Invalid values.** A loss that is not finite drops its micro-batch *before* the backward pass: once an invalid value reaches the optimiser’s moment estimates, they never recover. A micro-batch whose gradient is not finite is dropped as well.
- **Memory.** A micro-batch that runs out of memory is retried once with gradient checkpointing, which recomputes every layer in the backward pass instead of storing its intermediate values: the result is identical and the cost is roughly one extra forward pass. Only if that also fails is the micro-batch dropped, and its name recorded, since the scenes that fail this way are the largest ones and dropping them silently would bias training towards small objects.

- **Initial check.** An untrained equivariant network does not produce random rotations: its error on a fragment is the deterministic angle of its untrained frame, Equation (43). Before a run, the initial losses are compared with wide bands around the chance levels of Table 5, and the equivariance properties are tested.

15.6 Configuration

Table 6 lists the settings (I: intra layer, C: cross layer). The right-hand column gives the settings of the run whose learning-rate schedule is shown in Figure 16, where they differ from the defaults and are recorded.

Table 6: Training configuration.

setting	default	reported run
<i>data</i>		
subset, split	Everyday, official object split	
break patterns per object per epoch	8	
fracture labels for tokens and candidates	dihedral, $\tau = 0.9$	
tokens per scene T	2,048	
normalisation	one divisor per scene	
<i>model</i>		
channels C	64	128
heads $\mathcal{H}$; d_h (intra); d_k (cross)	4; 8; 16	
embedding size	32	
layer sequence	I, I, I, I, I, C, C, C, I	I, I, C, I, C, I, C, I
parameters	928,640	2,771,424
<i>objective</i>		
rotation target	anchor	
loss weights	all 1	
<i>optimisation</i>		
scenes per forward pass	1	
micro-batches per optimiser step K	2	16
optimiser steps per epoch	800	20
epochs	40	600
peak learning rate $\eta_{\max}$	10^{-3}	5×10^{-4}
warm-up	3% of training	none
floor $\eta_{\min}$	2×10^{-5}	6×10^{-5}
weight decay λ	10^{-4}	10^{-5}
gradient clipping (global norm)	1.0	
precision	32-bit	
GPUs	all available	1
validation scenes per epoch	all	128

16 Design decisions and their reasons

Table 7: The main design decisions.

decision	reason
learn rotations, solve translations	once orientations are known, placement is linear least squares; learning it would spend data on what geometry solves
equivariance by construction	the label transforms exactly like an equivariant prediction, so the error is independent of the perturbation and no rotation needs to be learned (Equation (17))
vector neurons	equivariance with plain linear algebra on 3-vectors: no spherical harmonics, no special libraries, moderate cost
every vertex a node, the mesh its graph	nothing has to be built inside a fragment; the mesh already encodes adjacency
edges carry two ordered face normals and an offset	the dihedral shape of the surface and the direction of the neighbour; the order is fixed by a rotation-invariant sign
one scale divisor per scene; log-radius through a gate	mating surfaces keep their size; scale has no direction, so it can only gate
equivariant to own pose, invariant to the others'	a fragment's label ignores the other fragments' perturbations, which are pure noise (Equation (18))
cross-fragment scores from invariants; matrix-valued messages	vector inner products across fragments depend on the relative perturbation; a sender may contribute an operator, never a direction
fixed token budget on dihedral-labelled vertices	bounds the cost of a cross layer by T^2 pairs; the dihedral rule works on one fragment and is identical at training and inference
intra layers after cross layers	cross layers change only tokens; the head pools every vertex
Gram-Schmidt frame, prediction $M^\top$	continuous, equivariant, proper by construction; the transpose follows from the convention (Equation (42))
anchor target	the absolute frame of an unseen, round object is unobservable; relative to the anchor, the network must only map all fragments to one common frame (Equation (47))
geodesic angle through atan2	arccos has an infinite derivative at zero error
contrastive embedding loss	a pull-only loss is minimised by collapse
all loss weights 1	the scales differ by nature; measure them before tuning
fragment-weighted accumulation	every fragment counts once, whatever the scene and the number of GPUs
learning rate as a function of the step	exact resumption, no rise after the end of the schedule
single precision	attention products overflow in half precision

A The chance level of the geodesic error

If R is uniformly distributed on $\text{SO}(3)$ (Haar measure), its rotation angle $\theta \in [0, \pi]$ has the density

$$p(\theta) = \frac{1 - \cos \theta}{\pi}, \quad (67)$$

which follows from the Weyl integration formula: in axis-angle coordinates the Haar measure has a density proportional to $\sin^2(\theta/2) = (1 - \cos \theta)/2$, uniform over the axis. Its mean is

$$\begin{aligned} \mathbb{E}[\theta] &= \frac{1}{\pi} \int_0^\pi \theta (1 - \cos \theta) d\theta = \frac{1}{\pi} \left(\frac{\pi^2}{2} - [\theta \sin \theta + \cos \theta]_0^\pi \right) \\ &= \frac{1}{\pi} \left(\frac{\pi^2}{2} + 2 \right) = \frac{\pi}{2} + \frac{2}{\pi} \approx 2.2074 \text{ rad} = 126.48^\circ, \end{aligned} \quad (68)$$

and since $\mathbb{E}[\theta^2] = \frac{1}{\pi}(\frac{\pi^3}{3} + 2\pi) = \frac{\pi^2}{3} + 2$, its standard deviation is $\sqrt{\pi^2/12 - 4/\pi^2} \approx 0.646 \text{ rad} = 37.0^\circ$. A prediction that is independent of the label errs by the angle of $\hat{R}^\top R$, which is uniform if either factor is, so its expected error is 126.5° . The same holds under the anchor protocol, because $\hat{R}_a^\top \hat{R}_i$ and $R_a^\top R_i$ are then independent and the second is uniform.

B Closed forms of the position and normal terms

Let E rotate by θ about the unit axis $\mathbf{u}$. By Rodrigues' formula, $E\mathbf{x} = \mathbf{x} \cos \theta + (\mathbf{u} \times \mathbf{x}) \sin \theta + \mathbf{u}(\mathbf{u} \cdot \mathbf{x})(1 - \cos \theta)$. Split $\mathbf{x} = \mathbf{x}_\parallel + \mathbf{x}_\perp$ along and across $\mathbf{u}$. Then $E\mathbf{x} - \mathbf{x} = (\cos \theta - 1)\mathbf{x}_\perp + \sin \theta (\mathbf{u} \times \mathbf{x}_\perp)$, whose two parts are orthogonal and of lengths $(1 - \cos \theta)\|\mathbf{x}_\perp\|$ and $\sin \theta \|\mathbf{x}_\perp\|$, so

$$\|E\mathbf{x} - \mathbf{x}\|^2 = \|\mathbf{x}_\perp\|^2((1 - \cos \theta)^2 + \sin^2 \theta) = 4 \sin^2 \frac{\theta}{2} \|\mathbf{x}_\perp\|^2. \quad (69)$$

For a unit vector $\mathbf{n}$, $\mathbf{n} \cdot E\mathbf{n} = (\mathbf{n} \cdot \mathbf{u})^2 + (1 - (\mathbf{n} \cdot \mathbf{u})^2) \cos \theta$, hence

$$1 - \cos \angle(E\mathbf{n}, \mathbf{n}) = (1 - \cos \theta)(1 - (\mathbf{n} \cdot \mathbf{u})^2) = 2 \sin^2 \frac{\theta}{2} (1 - (\mathbf{n} \cdot \mathbf{u})^2). \quad (70)$$

C Why the cross layer pools before building the matrix

The message to token i could be written as a sum over partners of per-pair matrices. Building the matrix from the pooled context instead, $G_i = \text{MLP}_G[\mathbf{c}_i; \mathbf{s}_i]$ with $\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{v}_j$, needs one matrix per receiving token: with 2,048 tokens, $\mathcal{H} = 4$ heads and $C = 128$ channels, that is $2048 \times 4 \times 32 \times 32$ numbers, about 34 MB in single precision, against about 2,000 times as much, some 70 GB, for one matrix per pair at four million pairs. Because the context passes through a nonlinear network, the result is not a sum of per-pair matrices, but every ingredient that crosses from the senders is still invariant, which is all that Equation (18) requires.

References

- [1] C. Deng, O. Litany, Y. Duan, A. Poulenard, A. Tagliasacchi, L. Guibas. Vector Neurons: a general framework for SO(3)-equivariant networks. *ICCV*, 2021.
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio. Graph attention networks. *ICLR*, 2018.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. Attention is all you need. *NeurIPS*, 2017.
- [4] R. Wu, C. Tie, Y. Du, Y. Zhao, H. Dong. Leveraging SE(3) equivariance for learning 3D geometric shape assembly. *ICCV*, 2023.
- [5] S. Sellán, Y.-C. Chen, Z. Wu, A. Garg, A. Jacobson. Breaking Bad: a dataset for geometric fracture and reassembly. *NeurIPS Datasets and Benchmarks*, 2022.
- [6] S. Li, Z. Jiang, G. Chen, C. Xu, S. Tan, X. Wang, I. Fang, K. Zyskowski, S. P. McPherron, R. Iovita, C. Feng, J. Zhang. GARF: learning generalizable 3D reassembly for real-world fractures. arXiv:2504.05400, 2025.
- [7] Z. Wang, J. Chen, Y. Furukawa. PuzzleFusion++: auto-agglomerative 3D fracture assembly by denoise and verify. *ICLR*, 2025.
- [8] Y. Zhou, C. Barnes, J. Lu, J. Yang, H. Li. On the continuity of rotation representations in neural networks. *CVPR*, 2019.
- [9] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. *ICLR*, 2019.

-
- [10] A. van den Oord, Y. Li, O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018.
 - [11] F. Mezzadri. How to generate random matrices from the classical compact groups. *Notices of the AMS* 54(5), 2007.
 - [12] P. J. Huber. Robust estimation of a location parameter. *Annals of Mathematical Statistics* 35(1), 1964.
 - [13] N. Thomas, T. Smidt, S. Kearnes, L. Yang, L. Li, K. Kohlhoff, P. Riley. Tensor field networks: rotation- and translation-equivariant neural networks for 3D point clouds. arXiv:1802.08219, 2018.
 - [14] M. Geiger, T. Smidt. e3nn: Euclidean neural networks. arXiv:2207.09453, 2022.